Circuitscape.jl

October 28, 2026

Contents

Contents ii

I Home 1

1 Circuitscape Documentation 2

2 Quick Start Guide 3

2.1 Running a job . . . 3

2.2 Building a Circuitscape Job . . . 3

2.3 Citing Circuitscape . . . 4

2.4 Circuitscape.jl vs Circuitscape 4 (Python) . . . 5

3 Related Projects 6

4 Further Reading 7

II User Guide 8

5 How Circuitscape Works 9

III Inputs, Outputs and Options 16

6 Options and Flags 17

6.1 INI Argument Reference . . . 17

6.2 Configuration Validation . . . 20

6.3 Data Type . . . 21

6.4 Modeling Mode . . . 22

6.5 Resistance Map or Network/Graph . . . 22

6.6 Focal Nodes (Pairwise, One-to-All, and All-to-One Modes) . . . 22

6.7 Advanced Mode Options . . . 23

6.8 Output Options . . . 23

6.9 Calculation Options . . . 24

6.10 Mapping Options . . . 25

6.11 Optional Input Files . . . 25

6.12 Input Raster Format . . . 26

6.13 Text List File Format . . . 27

6.14 Include/Exclude File Format . . . 29

6.15 Output Files . . . 30

IV On Solvers and Computation Time 33

7 Computational Limitations, Speed, and Landscape Size 34

7.1 Memory Limitations . . . 34

7.2 Multi-threading . . . 34

7.3 Convergence Checks . . . 35

7.4 What Makes a Run Fast . . . 35

7.5 Default Solvers: CG+AMG and CHOLMOD . . . 37

7.6 GeoTIFF Support . . . 37
7.7 Pardiso Solver . . . 38
7.8 Apple Accelerate Solver . . . 38

V Calling Circuitscape from other Programs 40
8 Calling Circuitscape from Other Programs 41
8.1 From Julia . . . 41
8.2 From R . . . 42
8.3 From Python . . . 42
8.4 Downstream packages . . . 43

VI Logging Options 44
9 Logging Options 45
9.1 Log Level . . . 45
9.2 Suppressing Messages . . . 45
9.3 Logging to File . . . 45
9.4 Disabling Log Output . . . 45

VII Upgrading to 6.0 47
10 Upgrading to 6.0 48
10.1 Breaking changes . . . 48

VIII API Reference 49
11 API Reference 50
11.1 Public API . . . 50
11.2 Internals . . . 53

Part I

Home

Chapter 1

Circuitscape Documentation

Circuitscape is an open-source Julia program that uses circuit theory to model connectivity in heterogeneous landscapes. Its most common applications include modeling movement and gene flow of plants and animals, as well as identifying areas important for connectivity conservation. Circuit theory complements commonly-used connectivity models because of its connections to random walk theory and its ability to simultaneously evaluate contributions of multiple dispersal pathways. Landscapes are represented as conductive surfaces, with low resistances assigned to landscape features types that are most permeable to movement or best promote gene flow, and high resistances assigned to movement barriers. Effective resistances, current flow, and voltages calculated across the landscapes can then be related to ecological processes, such as individual movement and gene flow.

More detail about the underlying model, its parameterization, and potential applications in ecology, evolution, and conservation planning can be found in McRae (2006) and McRae et al. (2008).

More detail about implementation can be found in the Circuitscape In Julia paper.

A PDF version of this documentation is also available.

Chapter 2

Quick Start Guide

This is a quick start guide. If you're looking for basics on how to use Circuitscape, refer to the usage guide.

To run Circuitscape, you need to install Julia. At the Julia prompt, install the Circuitscape package by running the following code

```
using Pkg
Pkg.add("Circuitscape")
```

You can also install Circuitscape in a local project as well.

2.1 Running a job

A Circuitscape job is fully described by an INI file. This configuration file consists of file paths to data as well as flag values. A detailed list of all INI arguments can be found in the Inputs, Outputs and Options section. Examples can be found in the test folder. You can also use a built-in terminal UI to build Circuitscape jobs. For more on that, skip ahead to Building a Circuitscape Job.

If you do have your INI file handy, you can run the job by:

```
using Circuitscape
Circuitscape.compute("myjob.ini")
```

You can also run Circuitscape programmatically by passing a configuration dictionary:

```
using Circuitscape
cfg = Circuitscape.init_config()
cfg["habitat_file"] = "resistance_map.asc"
cfg["point_file"] = "focal_nodes.asc"
cfg["scenario"] = "pairwise"
cfg["output_file"] = "output/results.out"
Circuitscape.compute(cfg)
```

2.2 Building a Circuitscape Job

The builder is kicked off by calling `Circuitscape.start()`, from the Julia prompt or a script. It will build an INI file for you step by step, and either run the job directly or write the final INI file to a specified location. You can exit the builder at any time by hitting Ctrl+C.



Please note that this version of Circuitscape **does not** come with a GUI.

You can also manually write your own INI file by copying and pasting an example from the test folder and changing values as needed.

2.3 Citing Circuitscape

Please use the following to cite Circuitscape:

```
@article{hall2021circuitscape,
  title={Circuitscape in julia: empowering dynamic approaches to connectivity assessment},
  author={Hall, Kimberly R and Anantharaman, Ranjan and Landau, Vincent A and Clark, Melissa and
  ↪  Dickson, Brett G and Jones, Aaron and Platt, Jim and Edelman, Alan and Shah, Viral B},
  journal={Land},
  volume={10},
  number={3},
  pages={301},
  year={2021},
  publisher={Multidisciplinary Digital Publishing Institute}
}
```

2.4 Circuitscape.jl vs Circuitscape 4 (Python)

Circuitscape.jl is built entirely in the Julia language, offering significant performance advantages over the previous Python version (v4.0.5).

Faster and More Scalable

The benchmark below dates from 2021 and compares Circuitscape.jl 5.x with the Python version (v4.0.5), both using 16 parallel processes, on problems from the standard Circuitscape benchmark suite. It predates the 6.0 performance work (chunked thread scheduling, native raster I/O, the removal of several graph copies), which is described with measurements under *What Makes a Run Fast* in On Solvers and Computation Time.

These benchmarks were run on a Linux (Ubuntu) server machine with the following specs:

- Name: Intel(R) Xeon(R) Silver 4114 CPU
- Clock Speed: 2.20GHz
- Number of cores: 20
- RAM: 384 GB

From the benchmark, Circuitscape.jl is up to *4x faster* on 16 processes. However, the best performing bar in the chart is *Julia-CHOLMOD*, which uses a direct solver.

Solvers, Threads and Precision

Circuitscape ships with the iterative `cg+amg` solver (the default) and the direct CHOLMOD solver, and can use Apple Accelerate (macOS) or Intel MKL Pardiso through optional package extensions. Runs are parallelized with Julia threads (`julia -t N` together with `parallelize = True`). Single precision (`precision = single`) is supported: it is tested in CI for `cg+amg`, CHOLMOD and Accelerate with a precision-aware residual check, and trades memory for some accuracy. GeoTIFF input and output needs the optional ArchGDAL package; ESRI ASCII grids need nothing extra.

All of this, including guidance on choosing a solver and the accuracy note on single precision, is in On Solvers and Computation Time. The INI options are listed in Inputs, Outputs and Options, and users of Circuitscape 5.x should read Upgrading to 6.0.

Chapter 3

Related Projects

1. Omniscape.jl - Omnidirectional connectivity analysis built on top of Circuitscape
2. AlgebraicMultigrid.jl - Algebraic Multigrid methods in Julia. This is the default solver used in Circuitscape.

Chapter 4

Further Reading

- Beier, P., W. Spencer, R. Baldwin, and B.H. McRae. 2011. Best science practices for developing regional connectivity maps. Conservation Biology 25(5): 879-892
- Dickson B.G., G.W. Roemer, B.H. McRae, and J.M. Rundall. 2013. Models of regional habitat quality and connectivity for pumas (*Puma concolor*) in the southwestern United States. PLoS ONE 8(12): e81898. doi:10.1371/journal.pone.0081898
- McRae, B.H. 2006. Isolation by resistance. Evolution 60:1551-1561.
- McRae, B.H. and P. Beier. 2007. Circuit theory predicts Gene flow in plant and animal populations. Proceedings of the National Academy of Sciences of the USA 104:19885-19890.
- McRae, B.H., B.G. Dickson, T.H. Keitt, and V.B. Shah. 2008. Using circuit theory to model connectivity in ecology and conservation. Ecology 10: 2712-2724.
- Shah, V.B. 2007. An Interactive System for Combinatorial Scientific Computing with an Emphasis on Programmer Productivity. PhD thesis, University of California, Santa Barbara.
- Shah,V.B. and B.H. McRae. 2008. Circuitscape: a tool for landscape ecology. In: G. Varoquaux, T. Vaught, J. Millman (Eds.). Proceedings of the 7th Python in Science Conference (SciPy 2008), pp. 62-66.
- Spear, S.F., N. Balkenhol, M.-J. Fortin, B.H. McRae and K. Scribner. 2010. Use of resistance surfaces for landscape genetic studies: Considerations of parameterization and analysis. Molecular Ecology 19(17): 3576-3591.
- Zeller K.A., McGarigal K., and Whiteley A.R. 2012. Estimating landscape resistance to movement: a review. Landscape Ecology 27: 777-797.

Part II

User Guide

Chapter 5

How Circuitscape Works

Whatever software you use, connectivity modeling involves a great deal of research, data compilation, GIS analyses, and careful interpretation of results. Defining areas to connect, parameterizing resistance models, and other modeling decisions you will need to make are not trivial. Before diving in, we strongly recommend that users first acquaint themselves with the process and challenges of connectivity modeling by consulting published resources. Good places to start include overviews on the Corridor Design and Connecting Landscapes websites. Spear et al. (2010), Beier et al. (2011) and Zeller et al. (2012) offer helpful advice on resistance mapping and connectivity analysis in general. Before using this software, users should acquaint themselves with the use of circuit theory for modeling connectivity (summarized in McRae et al. 2008).

See the Gnarly Landscape Utilities website for tools that can help to automate resistance and core area modeling.

Lastly, users interested in mapping important connectivity areas may wish to consider Linkage Mapper, which maps least-cost corridors. Linkage Mapper now also hybridizes least-cost corridor modeling with Circuitscape (see the Pinchpoint Mapper tool within the Linkage Mapper toolkit). Links to other connectivity tools can be found on the Corridor Design and Connecting Landscapes websites.

Circuitscape is run from Julia: `Circuitscape.compute("job.ini")` runs a job described by an INI file, `Circuitscape.compute(dic` runs one built in code, and the interactive INI builder `Circuitscape.start()` writes an INI file step by step and can run it. It can also be called from R or Python; see Calling Circuitscape from Other Programs. There is no graphical user interface or ArcGIS toolbox. Users supply Circuitscape with resistance data and the program calculates effective resistances and/or creates maps of current flow and voltages across landscapes and networks.

Two data types: network and raster

Circuitscape reads either a network of nodes connected by links or a raster grid of resistances (Fig. 3). Links and raster cells are attributed with resistance values that reflect the degree to which the landscape facilitates or impedes movement. Networks and raster maps can be coded in resistances (with higher values denoting greater resistance to movement) or conductances (the reciprocal of resistance; higher values indicate greater ease of movement).

Fig. 1. Simple illustrations of network and raster data types used by Circuitscape. The program can operate on networks of nodes (left panel) or raster grids (right panel). Raster grid cells can have any resistance value. Here, cells with zero resistance ("short-circuit regions," which can be used to represent contiguous habitat patches) are shown in white, cells with a resistance of 1 are shown in gray, and a cell with infinite resistance (coded as NODATA) is shown in black.

For rasters, every grid cell with finite resistance is represented as a node in a graph, connected to either its four first-order or eight first- and second-order neighboring cells. Cells with infinite resistance (zero conductance)

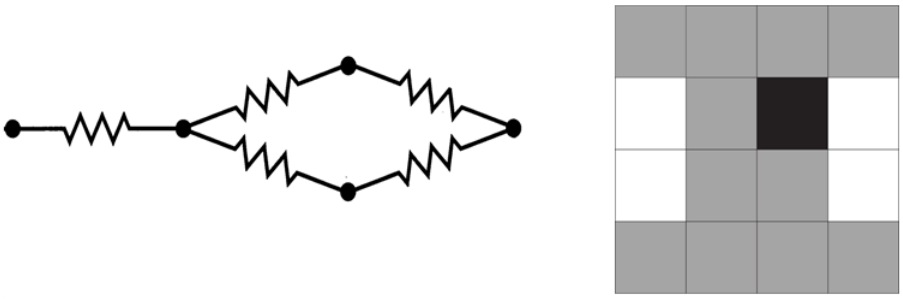


Figure 5.1:

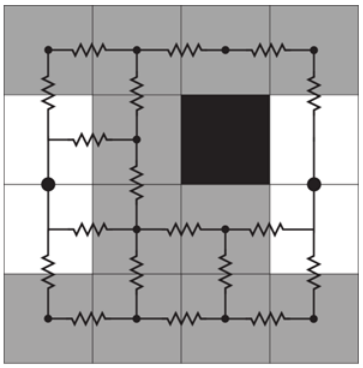


Figure 5.2:

are dropped. Habitat patches, or collections of cells, can be assigned zero resistance (infinite conductance) using a separate "short-circuit region" file. These collections of cells are collapsed into a single node.

Fig. 2. Raster grids are converted to electrical networks. Each cell becomes a node (represented by a dot), and adjacent cells are connected to their four or eight neighbors by resistors. Here, the two short-circuit regions have each been collapsed into a single node. The infinite resistance cell is dropped entirely from the network.

Calculation modes

Circuitscape operates in one of four modes: **pairwise**, **advanced**, **one-to-all**, and **all-to-one**. Pairwise and advanced modes are available for both raster and network data types. The one-to-all and all-to-one modes are available for raster data only.

In the **pairwise** mode, connectivity is calculated between all pairs of focal nodes (points or regions between which connectivity is to be modeled) supplied to the program in a single input file. For each pair of focal nodes, one node will arbitrarily be connected to a 1-amp current source, while the other will be connected to ground. Effective resistances will be calculated iteratively between all pairs of focal nodes, and, if selected, maps of

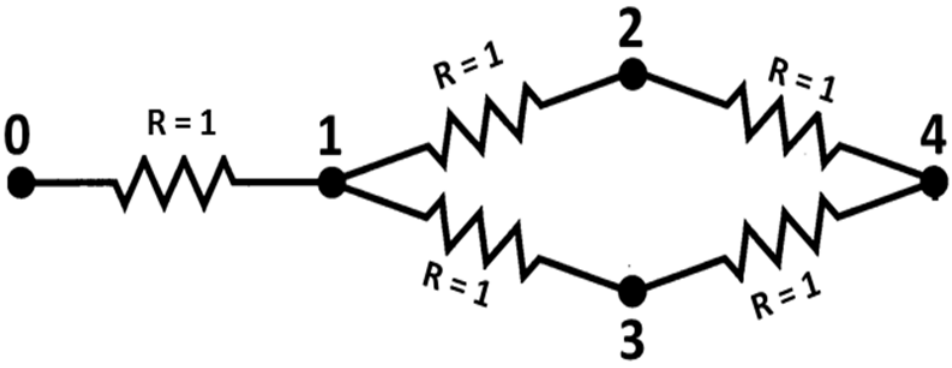


Figure 5.3:

current and voltage will be produced. If there are *n* focal nodes, there will be *n*(*n* - 1)/2 calculations **unless** you're using focal points (only one cell per focal node) and not mapping currents or voltages. In the latter case, we can do it in *n* calculations **(much faster)**.

The **advanced** mode offers much greater flexibility in defining sources and targets for current flow. The user defines any number of current sources and any number of grounds in a network or raster landscape, and these are all activated simultaneously. Sources represent points or areas from which current flows, whereas grounds represent nodes where current exits the system.

Source nodes can have different strengths (i.e. inject more or less current into the network or grid), and ground nodes can be tied to ground with any resistance. Current sources and grounds are supplied in separate input files.

Two other modes are available for raster data types only. The **one-to-all** mode is similar to the pairwise mode, and takes the same input files. However, instead of iterating across all focal node *pairs*, this mode iterates across all focal nodes. In each iteration, one focal node is connected to a 1-amp current source, while all remaining focal nodes are connected to ground. If there are *n* focal nodes, there will be *n* calculations.

The **all-to-one** mode is similar to the one-to-all mode, and takes the same input files. However, in this mode Circuitscape connects one focal node to ground and all remaining focal nodes to 1-amp current sources. It then repeats the process for each focal node; if there are *n* focal nodes, there will be *n* calculations.

Circuitscape can generate maps showing the current density and voltage at each node or cell under each configuration (and current flow for each link/resistor in networks). Additionally, Circuitscape writes a file reporting effective resistances between all pairs of focal nodes in the pairwise mode, and between each node and ground in the one-to-all mode. Resistances in the all-to-one mode are undefined, so a file is written with zeros indicating successful solves.

Illustrations of analyses with network data

For network data types, any node can be connected to any other node by a resistor:

Fig. 3. Example network. This network would be input as a **text list** specifying resistances between each pair of connected nodes (0-1, 1-2, 1-3, 2-3, and 2-4; see the *Input file formats* section below).

For **pairwise analysis** we would also supply a list of focal nodes (containing at least two node numbers, but as many as five, the number of nodes in the circuit) between which we want to perform calculations.

Fig 4. In pairwise mode, Circuitscape will iterate across pairs of nodes in a focal node list. If node 0 and node 4 are in the focal node list, then one of the iterations will look like the above, with a 1 amp current source

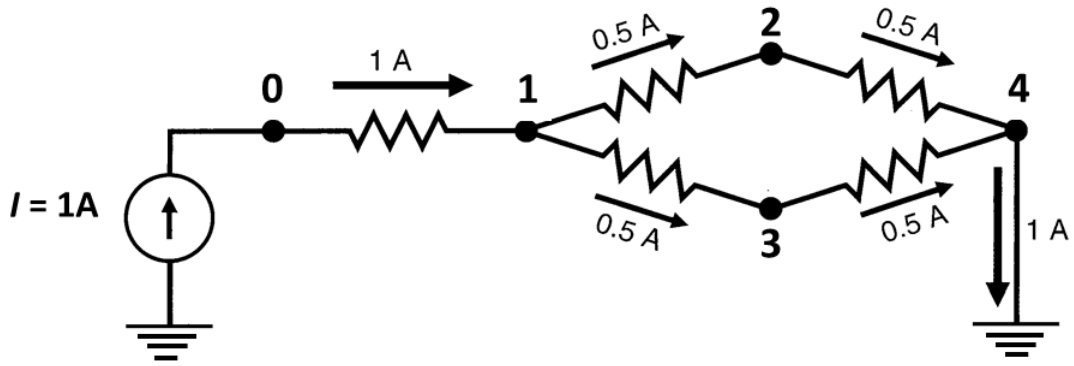


Figure 5.4:

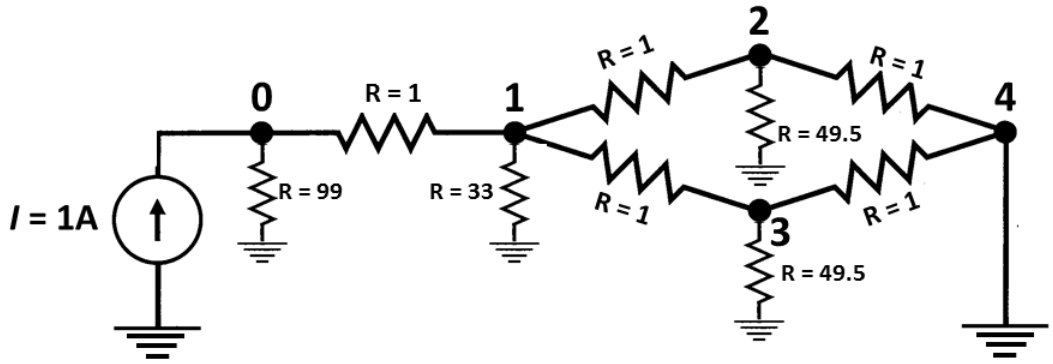


Figure 5.5:

connected to one node and the other grounded. Current will flow across the network from the source to the ground. Branch currents, node currents, node voltages, and effective resistances between node pairs can be written for each iteration.

More complexity can be added by running in **advanced mode**, which allows any number of sources and grounds to be activated simultaneously. For example, we could modify the circuit above by adding a single, fixed source at node zero and adding multiple grounds with different resistances. Current sources and grounds are entered in separate files.

Fig. 5. In advanced mode, any node can be tied to a current source or to ground, either directly or via resistors with any value (top panel). Currents passing through all nodes and links can then be calculated (bottom panel), and voltages can be calculated at each node. Circuit above is from McRae et al. (2008).

Illustrations of analyses with raster data

Fig. 6. Example raster input files for **pairwise, one-to-all, and all-to-one modes**. Input files in this example include a **resistance map** specifying per-cell resistances or conductances, a **focal node location file** (with

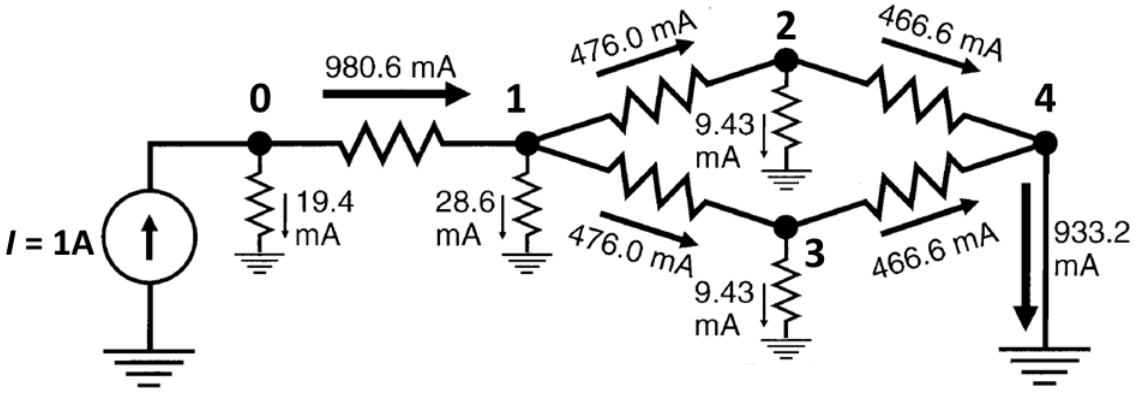


Figure 5.6:

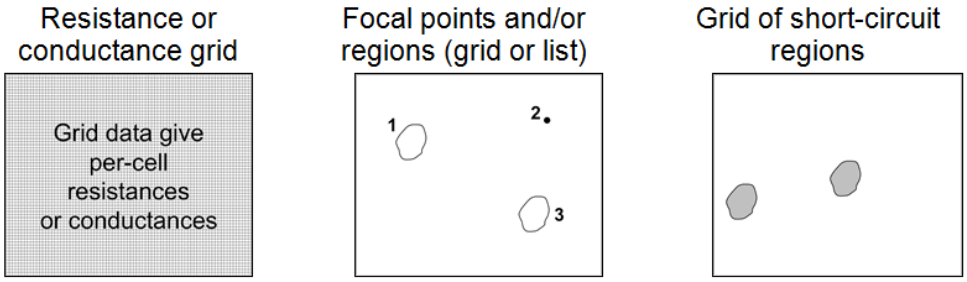


Figure 5.7:

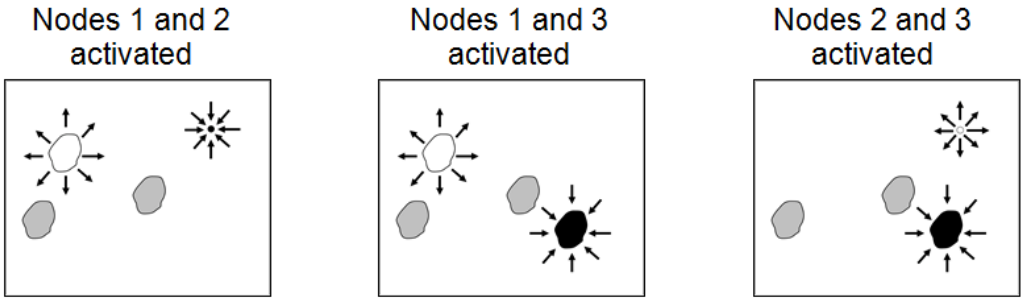


Figure 5.8:

two focal regions and one focal point in this case), and an optional **short-circuit region map**. Focal regions and short-circuit regions represent areas with zero resistance. Cells with the same region ID are considered perfectly connected and are collapsed into a single node, even if they are not contiguous.

Fig. 7. Schematic describing **pairwise** mode analyses that would result from the input files shown in Fig. 8. Three sets of pairwise calculations, involving focal nodes 1 and 2, nodes 1 and 3, and nodes 2 and 3, would be conducted. For each pair, one node would be connected to a 1-amp current source, and the other to ground.

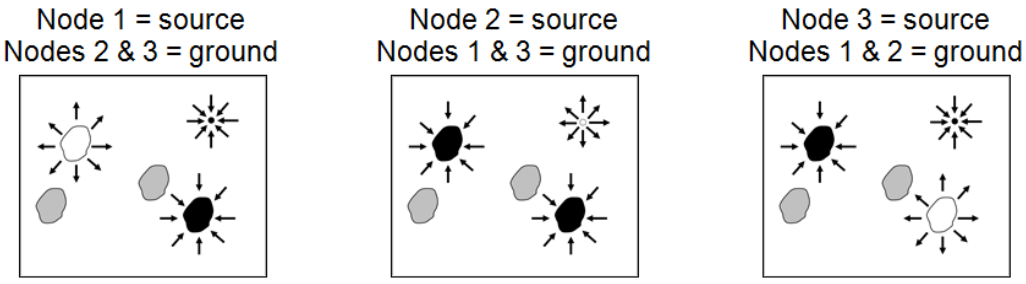


Figure 5.9:

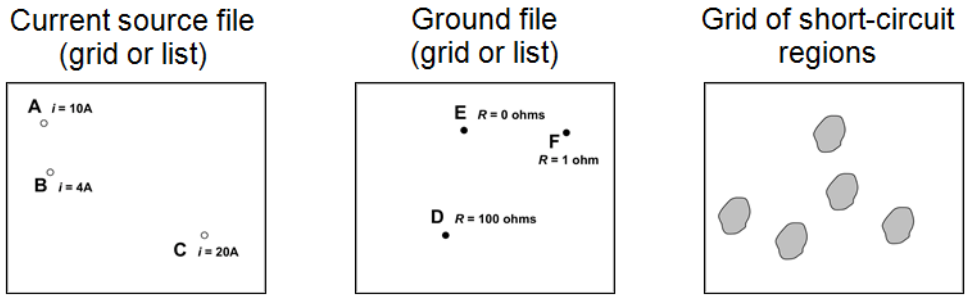


Figure 5.10:

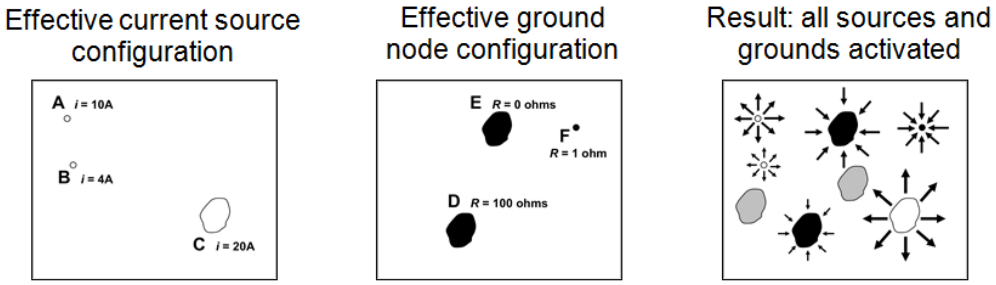


Figure 5.11:

Note that focal region nodes become short-circuit regions when they are activated (e.g., node 1 in scenario 1), but these regions are not present when the nodes are not activated (e.g., node 1 in scenario 3).

Fig. 8. Schematic describing **one-to-all mode** analyses that would result from the input files shown in Fig. 8. Three sets of calculations, involving focal nodes 1, 2, and 3, would be conducted. For each, one node would be connected to a 1-amp current source, and the other two would be connected to ground. The all-to-one mode is similar, with arrow directions reversed; that is, one node is connected to ground while the remaining nodes are connected to 1-amp current sources.

Fig. 9. Example raster input files for **advanced mode**, which requires independent **current source and ground files**. Note that current sources in this example have different "strengths," and ground nodes are connected to ground with differing levels of resistance. This example also includes an optional grid with five short-circuit regions.

Fig. 10. The first two panels show the "effective" configuration resulting from the input files in Fig. 11. Because current source C and grounds D and E overlap with short-circuit regions, these short-circuit regions effectively become sources or grounds themselves. The rightmost panel shows a schematic of the resulting analysis, with all sources (white points and polygons) and grounds (black points and polygons) activated simultaneously. Note that sources may be negative (drawing current out of the system), and ground nodes can actually contribute current to the system when negative sources are present.

Part III

Inputs, Outputs and Options

Chapter 6

Options and Flags

6.1 INI Argument Reference

All Circuitscape configuration is done through an `.ini` file. Below is a complete reference of all arguments, organized by section, with their types, default values, and descriptions.

Circuitscape Mode

Argument	Type	Default	Description
`data_type`	String	`raster`	Input data type. Values: `raster`, `network`.
`scenario`	String	`not entered`	Modeling mode. Values: `pairwise`, `advanced`, `one-to-all`, `all-to-one`.

Habitat Raster or Graph

Argument	Type	Default	Description
`habitat_file`	Path	—	Path to the resistance/conductance map file (raster or network). ESRI ASCII grids (`.asc`, optionally gzipped) are read natively; GeoTIFF and other GDAL formats require `using ArchGDAL` (see *GeoTIFF Support*).
`habitat_map_is_resistances`	Boolean	`True`	If `True`, habitat map values are resistances. If `False`, values are conductances.

Connection Scheme for Raster Habitat Data

Argument	Type	Default	Description
`connect_four_neighbors_only`	Boolean	`False`	If `True`, connect each cell to 4 cardinal neighbors only. If `False`, connect to all 8 neighbors.
`connect_using_avg_resistances`	Boolean	`False`	If `True`, use average resistance for cell connections. If `False`, use average conductance.

Argument	Type	Default	Description
`use_polygons`	Boolean	`False`	If `True`, read a short-circuit region file. Cells within each region are collapsed into a single node with zero resistance.
`polygon_file`	Path	—	Path to the short-circuit region map. Must have the same cell size and extent as the resistance grid.

Short-Circuit Regions (Polygons)

Options for Advanced Mode

Argument	Type	Default	Description
`source_file`	Path	—	Path to the current source file (raster or text list). Values specify source strength in amps.
`ground_file`	Path	—	Path to the ground point file (raster or text list). Values specify resistance or conductance to ground.
`ground_file_is_resistances`	Boolean	`True`	If `True`, ground file values are resistances. If `False`, conductances. Set to `False` with value 0 to connect directly to ground.
`use_unit_currents`	Boolean	`False`	If `True`, all current sources are set to 1 Amp regardless of values in the source file.
`use_direct_grounds`	Boolean	`False`	If `True`, all ground nodes are tied directly to ground (R=0) regardless of values in the ground file.
`remove_src_or_gnd`	String	`keepall`	When a source and ground are at the same node: `keepall`, `rmvsrc`, `rmvgnd`, or `rmvall`.

Options for Pairwise, One-to-All, and All-to-One Modes

Argument	Type	Default	Description
`point_file`	Path	—	Path to focal node file (raster or text list). Each focal node must have a unique positive integer ID.
`use_included_pairs`	Boolean	`False`	If `True`, only run calculations on a subset of focal node pairs specified in the included pairs file.
`included_pairs_file`	Path	—	Path to file specifying pairs to include or exclude from calculations.

Options for One-to-All and All-to-One Modes

Argument	Type	Default	Description
`use_variable_source_strengths`	Boolean	`False`	If `True`, read per-node source strengths from a file instead of using 1 Amp for all.
`variable_source_file`	Path	`None`	Path to text file with focal node IDs and corresponding source strengths.

Argument	Type	Default	Description
`use_mask`	Boolean	`False`	If `True`, apply a mask to the resistance map. Cells with negative, zero, or NODATA values in the mask are dropped.
`mask_file`	Path	`None`	Path to the raster mask file.

Mask File

Output Options

Argument	Type	Default	Description
`output_file`	Path	—	Base path and filename for all output files.
`write_cur_maps`	Boolean	`False`	If `True`, write current maps for each iteration and a cumulative map.
`write_volt_maps`	Boolean	`False`	If `True`, write voltage maps for each iteration.
`write_cum_cur_map_only`	Boolean	`False`	If `True`, calculate current maps for each iteration but only write the cumulative (summed) map to disk.
`write_max_cur_maps`	Boolean	`False`	If `True`, write a map showing the maximum current value at each cell across all iterations.
`set_null_currents_to_zero`	Boolean	`False`	If `True`, cells that are NODATA in the resistance map (and so have no graph node) are written as NODATA in current maps instead of 0.
`set_null_voltages_to_zero`	Boolean	`False`	If `True`, cells that are NODATA in the resistance map are written as NODATA in voltage maps instead of 0.
`set_focal_node_currents_to_zero`	Boolean	`False`	If `True`, the cells of the *active* focal nodes carry zero current in each raster current map: the two focal nodes of the pair in pairwise mode, every focal node in one-to-all and all-to-one. Cumulative and maximum maps then show only current that flows *through* a focal region while other pairs are active. Raster current maps only; network node currents and resistances are unaffected.
`compress_grids`	Boolean	`False`	Accepted for compatibility with Circuitscape 4. Output grids are currently written uncompressed; gzipped *input* grids (`.asc.gz`) are read.
`log_transform_maps`	Boolean	`False`	If `True`, log10-transform values in output current maps. Cells with zero current are set to NODATA.
`write_as_tif`	Boolean	`False`	If `True`, write current and voltage maps as GeoTIFF (LZW-compressed) instead of ESRI ASCII grids. Requires the optional GDAL support: `Pkg.add("ArchGDAL")`, then `using ArchGDAL` before running.

Calculation Options

Reclassification

Logging

The Circuitscape 4 keys `profiler_log_file`, `print_timings`, `print_rusages` and `screenprint_log` are accepted without a warning and have no effect.

Argument	Type	Default	Description
`solver`	String	`cg+amg`	Linear solver to use. Values: `cg+amg` (iterative, recommended for large grids), `cholmod` (direct, uses more memory), `accelerate` (direct, macOS only, requires AppleAccelerate.jl), `pardiso` (direct, requires Pardiso.jl). Also accepted: `amg+cg`, `cholesky`, `cholfact`, `mklpardiso`, `apple_accelerate`. Any other value is an error.
`precision`	String	`double`	Floating-point precision. Values: `double`, `single` (either case). Single precision is supported and tested for `cg+amg`, `cholmod` and `accelerate`; `pardiso` is double-only and switches to double with a warning. See the accuracy note under *Convergence Checks* in On Solvers and Computation Time.
`use_64bit_indexing`	Boolean	`False`	If `True`, always use 64-bit integer indices for node numbers and the sparse matrices. If `False`, 32-bit indices are used whenever the input fits (fewer than 100 million cells or network nodes with `cg+amg`, fewer than 8 million with the direct solvers whose Cholesky factor must fit 32-bit indices too, and a network's twice-edge count below `2^31`), and 64-bit indices otherwise. 32-bit indices halve the memory of the graph's index arrays, the connected components, the AMG hierarchy and the index arrays of the CHOLMOD factor; the results are identical.
`cholmod_batch_size`	Integer	`32`	Number of right-hand sides the direct solvers (`cholmod`, `pardiso`, `accelerate`) solve at once in pairwise mode. Postprocessing of each batch runs across threads. Ignored by `cg+amg`.
`residual_tolerance`	Float	`auto`	Every linear solve is accepted only if its true relative residual `‖Gv − b‖ / ‖b‖` is below this. `auto` means `1e-4` in double precision and `1e-3` in single. With `cg+amg`, a solve that stops short is retried with tighter Krylov tolerances before the run is aborted.
`parallelize`	Boolean	`False`	If `True`, run pair solves (or per-focal-node solves) in parallel using Julia threads. Start Julia with `julia -t N` for N threads; see *Parallelism* below.
`low_memory_mode`	Boolean	`False`	Circuitscape 4 option, not implemented in Circuitscape.jl. Setting it to `True` is an error so that it cannot be relied on silently.
`preemptive_memory_release`	Boolean	`False`	Circuitscape 4 option, not implemented in Circuitscape.jl. Setting it to `True` is an error.

6.2 Configuration Validation

Since 6.0 a configuration is checked in full before any data is read, and a problem is reported as an error rather than silently turning into a different run.

Values. `solver`, `scenario`, `data_type`, `precision`, `log_level`, `remove_src_or_gnd` and every boolean option accept only the values listed in the tables above. Anything else is an error naming the key and the valid values. In earlier versions `solver = cholmodd` ran `cg+amg`, `scenario = pairwsie` ran pairwise and `write_cur_maps = Ture` wrote nothing; each of these is now refused:

```
ArgumentError: solver = "cholmodd" is not recognised; expected one of: cg+amg, cholmod, pardiso,
↪  accelerate
ArgumentError: write_cur_maps = "Ture" is not recognised; expected one of: True, False
```

A missing `scenario` is an error (`scenario is not set; expected one of: pairwise, advanced, one-to-all, all-to-one`) rather than an implicit pairwise run.

Argument	Type	Default	Description
use_reclass_table	Boolean	false	If True, replace values in the habitat raster using the lookup table in reclass_file before the run. Raster mode only.
reclass_file	Path	—	Text file with two whitespace-separated columns, old new, one pair per line. Every cell whose value equals old is set to new; other cells, including NODATA, are left alone. Values are replaced in a single pass, so 1 2 followed by 2 3 sends 1 to 2, not to 3. Lets one large raster be reused with many resistance assignments without exporting a new file each time.

Argument	Type	Default	Description
log_file	Path	None	Path to log file. If None, no file logging.
log_level	String	INFO	Logging level. Values: DEBUG, INFO, WARNING, ERROR, CRITICAL (case-insensitive; WARN is also accepted; CRITICAL is treated as ERROR). Any other value is an error. DEBUG logs every pair solve and prints a timing table at the end of the run.
suppress_messages	Boolean	false	If True, suppress informational messages on the console. Warnings and errors are still shown, and a log_file still receives everything.

Files. Every input file the run will actually use must exist: habitat_file always; point_file in pairwise, one-to-all and all-to-one; source_file and ground_file in advanced mode; polygon_file, mask_file, included_pairs_file, variable_source_file and reclass_file only when the matching use_* option is True. The directory of output_file must exist. An INI Builder placeholder such as (Browse for a resistance file) counts as unset. All problems are collected and reported in one message, so a configuration is fixed in one round trip:

```
ArgumentError: Invalid configuration:
  - habitat_file = "resistance.asc" does not exist
  - point_file = "nodes.asc" does not exist
  - output directory "output" (from output_file) does not exist
  - low_memory_mode is not implemented in Circuitscape.jl; remove it or set it to False
```

Never-implemented options. low_memory_mode and preemptive_memory_release are Circuitscape 4 options that Circuitscape.jl parses but has never implemented. Setting either to True is an error, so that a large run cannot rely on them by mistake.

GeoTIFF without GDAL. If ArchGDAL is not loaded, a configuration that names a GeoTIFF input (detected by file content, not extension) or sets write_as_tif = True is refused up front with the installation hint (Pkg.add("ArchGDAL"), then using ArchGDAL). See *GeoTIFF Support* in On Solvers and Computation Time.

Unknown keys produce a warning (Ignoring unrecognised configuration keys: ...) and are otherwise ignored, so a misspelt key is visible without breaking old INI files.

Circuitscape 4 spellings still work: remove_src_or_gnd = not entered means keepall, Python log level names (WARNING, CRITICAL, any case) are read, and residual_tolerance = None means automatic.

6.3 Data Type

Set data_type to raster or network to choose whether you will be analyzing raster grid or network data.

6.4 Modeling Mode

Circuitscape is run in one of four modes (set via `scenario`). Pairwise and advanced modes are available for both raster and network data types. The one-to-all and all-to-one modes are available for raster data only.

6.5 Resistance Map or Network/Graph

The resistance file (`habitat_file`) specifies the ability of each cell in a landscape or link in a network to carry current. File formats are described in the *Input file formats* section below.

Most users code their data in terms of resistances (with higher values denoting greater resistance to movement). Set `habitat_map_is_resistances = False` to specify conductances instead (conductance is the reciprocal of resistance; higher values indicate greater ease of movement).

Zero and infinite values for conductances and resistances represent special cases. Infinite resistances are coded as NODATA values in input resistance grids, or as zero or NODATA in input conductance grids; these are treated as complete barriers and are disconnected from all other cells. For raster analyses, cells with zero resistance (infinite conductance) can be specified using a separate short-circuit region file as described below.

6.6 Focal Nodes (Pairwise, One-to-All, and All-to-One Modes)

The focal node file (`point_file`) specifies locations of nodes between which effective resistance and current flow are to be calculated. **Each focal node should have a unique positive integer ID.** Files may be text lists specifying coordinates or raster grids. When a grid is used, it must have the same cell size and extent as the resistance grid. Cells that do not contain focal nodes should be coded with NODATA values.

For raster analyses, focal nodes may occur at points (single cells on the resistance grid) or across regions. For the latter, a single ID would occupy more than one cell in a grid or more than one pair of coordinates in a text list (and falling within more than one cell in the underlying resistance grid). Cells within a single region are collapsed into a single node. The difference from short-circuit regions is that a focal region will be "burned in" to the resistance grid only for pairwise calculations that include that focal node. Focal regions need not be contiguous. For large grids, focal regions may require more computation time. When calculating resistances on large raster grids and not creating voltage or current maps, focal points will run much more quickly.

Parallelism

Circuitscape uses Julia's native threads. Two things are needed: start Julia with threads (`julia -t N`, or set the `JULIA_NUM_THREADS` environment variable before starting Julia) **and** set `parallelize = True` in the INI file. With `parallelize = False`, the default, the run is serial however many threads Julia has. The log line `Starting up Circuitscape to use N threads in parallel` confirms both are in effect.

What runs in parallel depends on the mode and solver:

- **Pairwise with `cg+amg`** (raster and network): the pair solves. All pairs of a connected component are collected and split into equal-sized chunks, about four per thread, each solved on its own task with its own preconditioner workspace. Every pair costs about the same, so equal counts are equal work. This includes the default configuration with no maps requested, where the resistance shortcut solves one system per focal node rather than one per pair: those solves used to run on a single task and are now spread across the threads too.
- **Pairwise with a direct solver** (`cholmod`, `pardiso`, `accelerate`): the factorization is computed once per connected component and `cholmod_batch_size` right-hand sides are solved at a time inside the solver library; the postprocessing of each batch (node currents, map accumulation and writing) runs across threads.

- **One-to-all and all-to-one**: one task per focal node. Each is an independent solve; the per-node current maps are accumulated on the main thread afterwards.
- **Advanced mode** is a single solve per connected component and is not parallelized.

Julia 1.12 starts an interactive thread in addition to the default worker threads. Circuitscape schedules its tasks on the default thread pool only, so `Threads.nthreads()`, the number given by `-t`, is what matters and the interactive thread changes nothing. BLAS is set to a single thread at startup because the workload is sparse.

Set `log_level = DEBUG` to get a timing table at the end of the run with the `solve linear system` and `postprocess` time per task.

6.7 Advanced Mode Options

Current Source File

The source file (`source_file`) specifies locations and strengths, in amps, of current sources. Either a raster or a text list may be used. Rasters must have the same cell size, projection, and extent as the resistance grid, and cells that do not contain current sources should be coded with NODATA values. Current sources may be positive or negative (i.e., they may inject current into the grid or pull current out). Similarly, grounds may either serve as a sink for current or may contribute current if there are negative current sources in the grid.

Ground Point File

The ground file (`ground_file`) specifies locations of ground nodes and resistances or conductances of resistors tying them to ground. Either a raster or a text list may be used. Rasters must have the same cell size, projection, and extent as the resistance grid, and cells that do not contain grounds should be coded with NODATA values. If a direct (R = 0) ground connection conflicts with a current source, the ground will be removed unless `remove_src_or_gnd` is set to `rmvgnd` or `keepall`.

Set `ground_file_is_resistances = False` if your ground point file specifies connections to ground in terms of conductance instead. To tie cells directly to ground, keep `ground_file_is_resistances = True` and set values in the ground point file to zero.

Unit Currents and Direct Grounds

Set `use_unit_currents = True` to force all current sources to 1 Amp, regardless of the value specified in the source file. Set `use_direct_grounds = True` to tie all ground nodes directly to ground (R=0) regardless of the ground file values.

Source/Ground Conflicts

When a cell is connected both to a current source and to ground, `remove_src_or_gnd` determines the behavior: `keepall` (default), `rmvsrc`, `rmvgnd`, or `rmvall`. When using `keepall`, if a source is tied directly to ground (zero resistance), the ground connection will be removed.

6.8 Output Options

Current Maps

When `write_cur_maps = True`, current maps will be generated for every pair of focal nodes in pairwise mode, or for the source/ground configuration in advanced mode. Current maps have the same dimension as the input files, with values at each cell representing the amount of current flowing through the node. In pairwise mode,

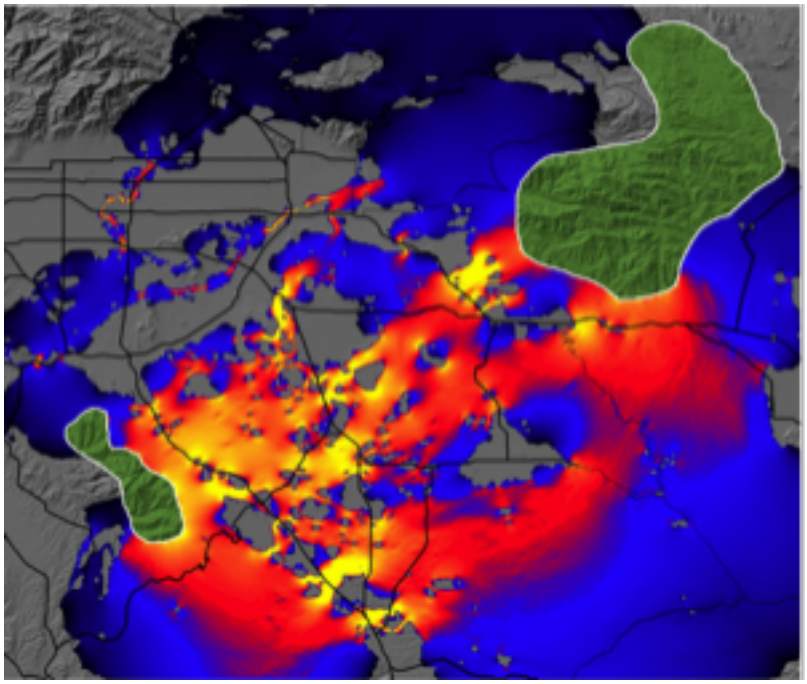


Figure 6.1:

a current map file will be created for each focal node pair, and a cumulative (additive) map will also be written. (Note that for a given pair of focal nodes, current maps are identical regardless of which node is the source and which is the ground due to symmetry.) For advanced mode, a single map will be written. Such maps can be used to identify areas which contribute most to connectivity between focal points (McRae et al. 2008).

Fig. 1. Current map used to predict important connectivity areas between core habitat patches (green polygons, entered as focal regions) for mountain lions. Warmer colors indicate areas with higher current density. "Pinch points," or areas where connectivity is most tenuous, are shown in yellow. Research Collaborators: Brett Dickson and Rick Hopkins, Live Oak Associates.

Voltage Maps

When `write_volt_maps = True`, voltage maps are written. In pairwise mode, these give node voltages observed for each focal node pair if one node were connected to a 1 amp current source and the other to ground. In advanced mode, voltage maps show voltages resulting from the source and ground configurations.

6.9 Calculation Options

Cell Connectivity

For raster operations, Circuitscape creates a graph by connecting cells to their neighbors. Set `connect_four_neighbors_only = True` for 4 cardinal neighbors only (default is 8, including diagonals).

Average Resistance vs Conductance

Set `connect_using_avg_resistances = True` to connect cells by their average resistance instead of average conductance (the default).

The distinction is particularly important when connecting cells with zero or infinite values. When average resistances are used, first-order neighbors are connected by resistors with resistance: $Rab = (Ra + Rb) / 2$, and second-order (diagonal) neighbors by: $Rab = \text{sqrt}(2) * (Ra + Rb) / 2$. When average conductances are used, first-order neighbors are connected by: $Gab = (Ga + Gb) / 2$, and second-order (diagonal) neighbors by: $Gab = (Ga + Gb) / (2 * \text{sqrt}(2))$.

6.10 Mapping Options

Maximum Current Maps

In pairwise, one-to-all, and all-to-one modes, current maps are created for every iteration. By default, Circuitscape writes a cumulative map showing the sum of values at each node or grid cell across all iterations. Set `write_max_cur_maps = True` to also write a map showing the maximum current value at each cell across iterations.

Cumulative Maps Only

Set `write_cum_cur_map_only = True` to calculate current maps for each iteration but only write the cumulative (and optionally maximum) map to disk. This saves disk space when many iterations are performed.

Compress Output Grids

Set `compress_grids = True` to compress output ASCII grids using gzip. This can be useful when many large maps will be written.

Log-Transform Current Maps

Set `log_transform_maps = True` to apply a log10 transform to current densities in output maps. Cells with zero current will be set to NODATA values.

Set Focal Node Currents to Zero

When `set_focal_node_currents_to_zero = True`, focal nodes have zero current in raster current maps when they are activated. For pairwise mode, cumulative maps will still show currents flowing through focal regions from other pairs being activated. This helps show the importance of each focal region for connecting others (see Dickson et al. 2013). In one-to-all and all-to-one modes every focal node is active on each solve, so all focal cells are zeroed. Applies to raster current maps; network node currents are unaffected.

6.11 Optional Input Files

Mask File

Set `use_mask = True` and provide `mask_file` to apply a raster mask. Cells with negative, zero, or NODATA values in the mask will be dropped from the resistance map (treated as complete barriers). Positive integer cells will be retained. File should only contain integers and be in raster format.

Short-Circuit Region Map

Set `use_polygons = True` and provide `polygon_file` to load short-circuit regions. These act as areas of zero resistance, providing patches through which current gets a "free ride." Each region should have a unique positive integer identifier; cells within each region are merged into a single node, including non-adjacent cells (regions need not be contiguous). Non-region areas should be stored as NODATA values. The file must have the same cell size and extent as the resistance grid.

Variable Source Strengths

In one-to-all and all-to-one modes, set `use_variable_source_strengths = True` and provide `variable_source_file` with focal node IDs and corresponding source strengths. The file should be a text list with two columns (ID followed by source strength). All nodes not in the list will default to 1 Amp.

Include/Exclude Focal Node Pairs

Set `use_included_pairs = True` and provide `included_pairs_file` to restrict calculations to a subset of focal node pairs. Users can either identify pairs to include or pairs to exclude, as specified in the first line of the file. This affects all modes except advanced mode. Files should be in tab-delimited text with a .txt extension.

6.12 Input Raster Format

Raster input maps should be stored in Arc/Info ASCII grid or GeoTIFF format, as exported by standard GIS packages. ASCII grids (optionally gzipped, with `xllcorner`/`yllcorner` or `xllcenter`/`yllcenter`, `cellsize` or dx/dy) are read natively; GeoTIFF needs the optional ArchGDAL package (see *GeoTIFF Support* in On Solvers and Computation Time). Set `write_as_tif = True` to produce GeoTIFF output instead of ASCII grids. For focal nodes, the value stored in each grid location refers to the focal node ID, and a single ID can occupy more than one cell (IDs must be positive integers). For current sources, the grid value specifies the source strength in amps. For grounds, the grid value specifies either the resistance or conductance of the resistor tying each ground node to ground.

The ASCII raster format is as follows:

Header:

```
ncols        <Number of columns>
nrows        <Number of rows>
xllcorner    <X coordinate of lower left corner>
yllcorner    <Y coordinate of lower left corner>
cellsize     <size of each cell>
NODATA_value <Code for cells with no habitat, focal nodes, sources or grounds>
```

Body (grid data):

Numeric data only. Columns are delimited with tabs and rows are delimited with new line characters.

Examples

Below is a 10 x 10 resistance map. Cells with infinite resistance are assigned NODATA values (-9999):

```
ncols         10
nrows         10
xllcorner     1
yllcorner     1
cellsize      1
NODATA_value  -9999
130    168    153    -9999  14     12    13     107    140    171
104    3      2      -9999  13     158   12     14     13     114
124    2      2      12     -9999  -9999 13     161    4      5
184    5      4      14     13     14    -9999  13     4      4
105    143    103    169    -9999  115   10     -9999  166    14
187    1      163    188    121    142   14     175    -9999  10
```

```
198    11     110    115    149    2      2      164    3      -9999
100    11     193    14     12     4      2      1      11     13
-9999  11     12     11     10     12     167    157    181    157
-9999  -9999  122    134    12     157    192    184    190    172
```

Below is a 10 x 10 focal region map. Groups of cells have been coded as focal regions that will be treated as "core area polygons" to be connected in circuit analyses. All cells within each focal region will be collapsed into a single node (even the non-contiguous cell in region #1) when that region is activated in pairwise, one-to-all, or all-to-one analyses. This format is identical to the short-circuit region file format.

```
ncols                10
nrows                10
xllcorner            1
yllcorner            1
cellsize             1
NODATA_value -9999
-9999  -9999  -9999  -9999  -9999  -9999  -9999  -9999  -9999  -9999
-9999  1      1      -9999  -9999  -9999  -9999  -9999  -9999  -9999
-9999  1      1      -9999  -9999  -9999  -9999  -9999  3      3
-9999  1      1      -9999  -9999  -9999  -9999  -9999  3      3
-9999  -9999  -9999  -9999  -9999  -9999  -9999  -9999  -9999  -9999
-9999  1      -9999  -9999  -9999  -9999  -9999  -9999  -9999  -9999
-9999  -9999  -9999  -9999   2      2     -9999  -9999  -9999  -9999
-9999  -9999  -9999  -9999   2      2      2     -9999  -9999  -9999
-9999  -9999  -9999  -9999  -9999  -9999  -9999  -9999  -9999  -9999
-9999  -9999  -9999  -9999  -9999  -9999  -9999  -9999  -9999   -9999
```

Note that regions 1 and 2 are well-connected by a low-resistance corridor in the resistance map above. Region 3 is connected to the other two regions only if cells are connected to their eight neighbors. In the four-neighbor case, region 3 would be completely isolated.

6.13 Text List File Format

For network/graph operations, resistor networks, focal nodes, current sources, and grounds should be stored as text lists (saved with a ".txt" extension). To specify a network of resistors, three columns are used. The first and second columns give the node IDs being connected by a resistor, and the third column gives the resistance value. For example, the simple circuit:

can be defined by the following text list:

```
    0    1    1
    1    2    1
    1    3    1
    2    4    1
    3    4    1
```

Please note: typically, there should just be one entry for each pair of connected nodes. If there are two entries for a single pair in the form of (node1, node2, value1) and (node2, node1, value2), these will be considered parallel resistors and their conductances will be summed. For example, if the above text list had an extra entry for node pair (4, 3) like this:

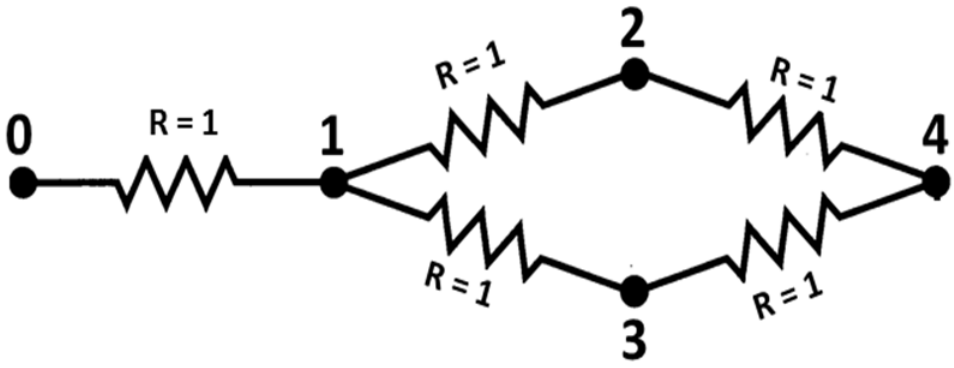


Figure 6.2:

```
0    1    1
1    2    1
1    3    1
2    4    1
3    4    1
4    3    1
```

then the resistance between nodes 3 and 4 in the resulting graph would be 1/2 ohm.

For advanced mode, current sources and grounds are also stored as text lists. The above circuit can be expanded to include a current source and grounds with two extra input files. For example, we can add a 1 Amp current source at node 0 with a file that looks like this:

```
0    1
```

To tie node 4 directly to ground (i.e. to connect it to ground with a wire that has a resistance of 0 Ohms) and connect the remaining nodes to ground with resistors, we can use a file that looks like this:

```
0    99
1    33
2    49.5
3    49.5
4    0
```

The resulting circuit would look like this (from McRae et al. 2008):

For **raster** operations, you can also store focal nodes, current sources, and grounds as text lists (saved with a ".txt" extension). For each node referenced in a text list, a value and X and Y coordinates are specified as shown below.

```
Value1 X1 Y1
Value2 X2 Y2
…
```

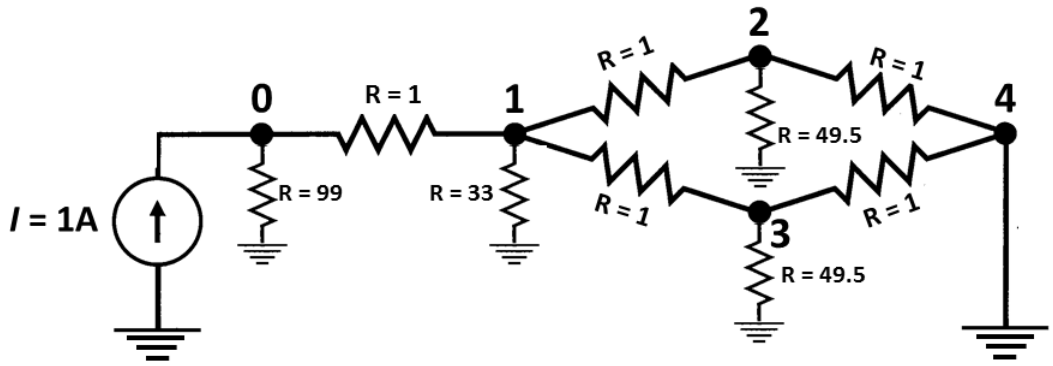


Figure 6.3:

Note: X and Y are geographical coordinates, not row and column numbers.

Example text list (a partial list of the cell locations in the focal region map above; coordinates are for cell centroids):

```
    1    2.5    9.5
    1    3.5    9.5
    1    2.5    8.5
    1    3.5    8.5
    1    2.5    7.5
    1    3.5    7.5
    1    2.5    5.5
    2    6.5    4.5
    ...
```

For focal nodes, the value field references the focal node ID; values must be positive integers, and a single ID can occupy more than one pair of coordinates (and more than one cell in the underlying resistance grid). For current sources, the value field references the source strength in amps. For grounds, the value field references either the resistance or conductance of the resistor tying each ground node to ground.

6.14 Include/Exclude File Format

This file is used when `use_included_pairs = True`, and affects all modes except advanced mode. There are two file formats that can be used. The first is the simplest, and gives a list of pairs to include in calculations, or pairs to exclude, as specified in the first line of the file. For example, if there are five focal nodes, numbered 1-5, and the following list is entered, only pairs (1,2), (1,3), and (1,5) will be analyzed:

```
mode    include
1        2
1        3
1        5
```

Similarly, if the first line in the above file read:

```
mode     exclude
```

all pairs except (1,2), (1,3), and (1,5) would be analyzed.

The second method uses a matrix identifying which pairs of focal nodes to connect. The file specifies minimum and maximum values in the matrix to consider a pair connected. This method can be useful when used with a distance matrix to only run analyses between points separated by a minimum distance, or by a distance equal to or less than a maximum distance. Note: any focal node not in the matrix will be dropped from analyses. Entries on the diagonal are ignored. For example, in the following matrix, only pairs with entries between 2 and 50 are connected. Pairs (1,2), (2,4), and (3,4) will not be analyzed. Focal node 5 will be dropped entirely:

```
min    2
max    50
0     1     2     3     4     5
1     0     100   6.67  7     1
2     100   0     11    1     60
3     6.67  11    0     -1    100
4     7     1     -1    0     0
5     1     60     100  0     0
```

Make sure to include a zero in the upper-left corner of the matrix.

Files should be in tab-delimited text with a .txt extension.

6.15 Output Files

Every output name starts with `output_file` with its `.out` extension removed; below this prefix is written `<out>`. The configuration as run is written, in INI form, to `output_file` itself, so the options are recorded next to the results. Focal node IDs in file names are the IDs from the focal node file.

Which files are written

Raster modes write ESRI ASCII grids (`.asc`), or GeoTIFF (`.tif`) when `write_as_tif = True`. When the input raster carries a projection it is written alongside each `.asc` as a `.prj` sidecar.

One-to-all and all-to-one do not write a resistances file; their result (one value per focal node) is the return value of `compute`. `set_focal_node_currents_to_zero`, `log_transform_maps` and `set_null_currents_to_nodata` are applied to each per-pair or per-node map before it is accumulated, so they affect the cumulative and maximum maps as well.

Network modes write tab-separated text. Branch current files omit branches carrying less than `1e-6` A.

`write_max_cur_maps` has no effect in network mode. Node current files have two columns (node ID, current); branch current files three (node, node, current); voltage files two (node ID, voltage).

Current and Voltage Data

Current and voltage data for networks are written in text list formats. Raster voltage and current maps are written in ESRI ASCII raster format (`.asc`), or GeoTIFF if `write_as_tif = True`. ASCII grids are read and written natively; when the input raster carries a projection, it is written alongside each `.asc` output as a `.prj` sidecar, as GIS tools expect.

File	Mode	Written when
<out>_resistances.out, <out>_resistances_3columns.out	pairwise	always
<out>_curmap_<i>_<j>.asc	pairwise	write_cur_maps = True and write_cum_cur_map_only = False
<out>_cum_curmap.asc	pairwise	write_cur_maps = True or write_cum_cur_map_only = True
<out>_max_curmap.asc	pairwise	write_max_cur_maps = True, together with write_cur_maps or write_cum_cur_map_only
<out>_voltmap_<i>_<j>.asc	pairwise	write_volt_maps = True
<out>_curmap_<id>.asc (one per focal node)	one-to-all, all-to-one	write_cur_maps = True or write_cum_cur_map_only = True (write_cum_cur_map_only does not suppress the per-node maps in these modes)
<out>_cum_curmap.asc, <out>_max_curmap.asc	one-to-all, all-to-one	as in pairwise
<out>_voltmap_<id>.asc	one-to-all, all-to-one	write_volt_maps = True
<out>_curmap.asc	advanced	write_cur_maps = True or write_cum_cur_map_only = True
<out>_voltmap.asc	advanced	write_volt_maps = True

Resistance Files

Resistance data are written in both matrix and 3-column formats.

Here are pairwise resistances written to the output directory for the eight neighbor case (using per-cell resistances and average resistances for cell connection calculations). The first row and column contain the focal node IDs:

```
0          1            2            3
1          0            11.93688471  15.03634473
2          11.93688471  0            11.57640568
3          15.03634473  11.57640568  0
```

Here are pairwise resistances written to the output directory for the four neighbor case, in which focal node 3 was completely isolated (-1 indicates infinite resistance):

```
0          1            2            3
1          0            33.55792693  -1
2          33.55792693  0            -1
3          -1           -1           0
```

For convenience, resistances are also written to a separate file in a 3-column format, e.g.:

File	Mode	Written when
`<out>_resistances.out`, `<out>_resistances_3columns.out`	pairwise	always
`<out>_node_currents_<i>_<j>.txt`, `<out>_branch_currents_<i>_<j>.txt`	pairwise	`write_cur_maps = True`. Since 6.0 these honour the flag as the raster path always did; before, they were written on every run. (They are also produced when any other map option is on, since every map option enables the per-pair postprocessing; `write_cum_cur_map_only` does not suppress them.)
`<out>_node_currents_cum.txt`, `<out>_branch_currents_cum.txt`	pairwise	`write_cur_maps = True`
`<out>_voltages_<i>_<j>.txt`	pairwise	`write_volt_maps = True`
`<out>_node_currents.txt`, `<out>_branch_currents.txt`	advanced	`write_cur_maps = True` or `write_cum_cur_map_only = True`
`<out>_voltages.txt`	advanced	`write_volt_maps = True`

```
1      2       33.55792693
1      3       -1
2      3       -1
```

Part IV

On Solvers and Computation Time

Chapter 7

Computational Limitations, Speed, and Landscape Size

We have tested this code on landscapes with up to 437 million cells. Increasing numbers of connections using diagonal (eight neighbor) connections will decrease the size of landscapes that can be analyzed. Also, increasing landscape size or numbers of focal nodes will increase computation time. Note that due to the matrix algebra involved with solving many pairs of focal nodes, Circuitscape will run much faster when focal points (each focal node falls within only one grid cell), rather than focal regions (at least one focal node occupies multiple grid cells), are used.

7.1 Memory Limitations

There are several ways to increase the solvable grid size:

- Set impermeable areas of your resistance map to NODATA
- Use focal points instead of regions in pairwise mode
- Connect cells to their four neighbors only (`connect_four_neighbors_only = True`)
- Disable current and voltage maps (`write_cur_maps = False`, `write_volt_maps = False`)
- Use the one-to-all or all-to-one modes, which typically use less memory and run more quickly than pairwise mode
- Use the `cg+amg` solver instead of `cholmod`, `accelerate`, or `pardiso` for large grids (direct solvers use significantly more memory)
- Coarsen your grids (use larger cell sizes) – this often produces qualitatively similar results (see McRae et al. 2008)

The all-to-one mode can be an alternative to pairwise mode when the goal is to produce a cumulative map of important connectivity areas among multiple source/target patches.

7.2 Multi-threading

Circuitscape uses Julia's native threading for parallel computation. Start Julia with multiple threads to take advantage of this:

```
julia -t 4    # use 4 threads
```

The cg+amg solver benefits most from threading — each focal point pair is solved independently on a separate thread. For problems with many focal points, this can provide significant speedups.

The cholmod, accelerate, and pardiso solvers perform batched direct solves. Threading in these modes parallelizes the postprocessing step (current map accumulation and output writing).

7.3 Convergence Checks

Every linear solve, whichever solver produced it, is accepted only if its true relative residual ||Gv − b|| / ||b|| is below residual_tolerance (by default 1e-4 in double precision and 1e-3 in single). For cg+amg there is a subtlety: Krylov's stopping test is on the *preconditioned* residual, and when the multigrid preconditioner is a poor fit for a particular component the two norms can disagree — CG reports success while the true residual is still too large. Circuitscape then tightens the Krylov tolerance and retries (up to three attempts) before failing, so a single awkward component does not abort a long run. The retry is logged as a warning; if you see many of them, the grid probably has extreme resistance contrasts worth examining.

Single precision (precision = single) halves memory at a cost in accuracy that depends on the problem: on the largest raster in the test suite, effective resistances differ from the double precision run by about 1%, and the gap will be larger on bigger grids or with stronger resistance contrasts. The difference does not change when residual_tolerance is tightened, so it is a property of the Float32 arithmetic rather than of solver convergence; verify against a double precision run before relying on single precision results.

7.4 What Makes a Run Fast

The facts below are measured; each cites the pull request that carries the full table. Timings were taken on the development machines named in those PRs and are for relative comparison only.

Circuitscape 5.17 vs 6.0

Three end-to-end runs of v5.17.1 (from the General registry) against 6.0 (master at c867d5a), on an Apple M2 Max with Julia 1.12.7, 8 threads (JULIA_NUM_THREADS=8, parallelize = True), solver cg+amg, double precision, eight-neighbour connectivity, random integer resistances 1–10, 16 single-cell focal points, ESRI ASCII inputs. Each cell is a fresh Julia process: one warm-up compute call, then one timed call (a single run, not a best-of-N). "Allocated" is Julia's total allocation for that call (@allocated); "peak RSS" is Sys.maxrss() at process end.

The default path is 4.9× faster because the resistance-shortcut solves now run in parallel (they were serial in 5.x) and the graph-construction and postprocessing overheads were removed. With maps on, the gain (2.1×, 39% less allocation) comes from computing node currents without sparse temporaries. One-to-all is essentially unchanged because its cost is the per-focal-point AMG solve, which 6.0 did not change. Peak RSS on the default path is flat because it is dominated by the AMG hierarchy for a 4M-cell Laplacian, which is identical in both versions. These are single runs on one machine and will vary; the per-PR tables in the sections below are the component measurements. Load time also dropped: using Circuitscape went from 2.71 s to about 1.2 s, and the resolved dependency count from 195 to 104 packages (ArchGDAL optional, Graphs removed).

Ask only for what you need

- **The resistance shortcut.** In raster pairwise mode, when no current, voltage, cumulative or maximum map is requested and no include/exclude file is used, Circuitscape solves one linear system per focal node of a component instead of one per pair, and derives every pairwise resistance from those voltages:

Case	Metric	v5.17.1	6.0
pairwise, default settings (no maps), 2000×2000 (4M cells)	time	72.3 s	14.9 s
	allo-cated	27.1 GiB	22.1 GiB
	peak RSS	9.7 GiB	9.9 GiB
pairwise + current maps, 1000×1000, all 119 pairs (include file defeats the shortcut)	time	61.5 s	28.7 s
	allo-cated	128.4 GiB	78.1 GiB
	peak RSS	5.7 GiB	4.1 GiB
one-to-all, 1000×1000	time	6.1 s	5.7 s
	allo-cated	45.5 GiB	42.4 GiB
	peak RSS	11.7 GiB	11.0 GiB

`n − 1` solves rather than `n(n − 1)/2`. The log says `Triggering resistance calculation shortcut` when it is active. Requesting any map, or an include/exclude file, switches to one solve per pair.

- **No maps, no postprocessing.** Since 6.0 (#500), when none of the four map options is set the per-pair postprocessing (node currents, a full-grid current map, the locked accumulation into a cumulative map) is skipped entirely. On a 400×400 grid, 16 focal points, 119 pairs with the shortcut defeated, 8 threads: 4.96 s → 3.31 s and 20.28 GiB → 3.49 GiB allocated per run. On a 1M-cell, 629-pair run this path had been 19% of profile samples and about 1 GB of allocation per pair.

Threads

Set `parallelize = True` and start Julia with threads (`julia -t N`). Since #491 the `cg+amg` pair solves are scheduled in equal-sized chunks, about four per thread, rather than one task per source node. That matters most for the default no-maps path: the shortcut anchors every solve at one source, so before 6.0 all of them ran on a single task whatever `-t` was. Measured on random resistance grids, 8 threads, best of 3:

Grid	Case	Before	After
400×400	shortcut (default), 16 focal points	2.37 s	0.64 s
400×400	all pairs, 16 focal points (119 pairs)	5.20 s	4.66 s
2000×2000 (4M cells)	shortcut (default), 16 focal points (15 solves)	73.9 s	20.4 s
2000×2000	all pairs, 9 focal points (35 solves)	59.6 s	48.2 s

Large all-pairs runs are memory-bandwidth bound, so extra threads help them less. In one-to-all and all-to-one each focal node is an independent task (#505 also stopped rebuilding the graph per focal node when an include file is used with single-cell focal points: 500×500 grid, 30 points, 5.33 GiB → 4.35 GiB allocated).

Memory

Load-time memory decides the largest grid that fits, and 6.0 removed several copies of the graph from that path:

Change	PR	Measured
ESRI ASCII grids read and written natively, no GDAL and no lock	#492	100 MB `.asc`: write 2.20 s → 0.58 s, read at parity with GDAL; files ~17% smaller
GDAL optional (ArchGDAL is a package extension)	#493	default install resolves 110 packages instead of 195 (104 once Graphs was removed in #495)
Connected components computed on the CSC structure; no Graphs adjacency copy	#495	2000×2000 grid: 1,574 MB → 140 MB allocated, 458 MB → 0 retained, 0.73 s → 0.32 s
`construct_graph` sizes its buffers exactly and emits both edge orientations; no `a + a'`	#504	2000×2000 grid: 2,536 MiB → 1,311 MiB allocated (−48%) for the same 0.5 GB matrix
Node currents in one sweep over the matrix, no sparse temporaries	#501	1000×1000 grid: 967 MB → 8 MB allocated per pair, about 10× faster
Network cumulative branch currents indexed once	#497	accumulation per pair is O(E) instead of O(E²)

Direct or iterative

- `cg+amg` (default) keeps memory proportional to the number of nonzeros in the Laplacian and parallelizes the pair solves. It is the only choice for the largest grids; a run with more than 5 million cells and `solver = cholmod` logs a warning to that effect.
- The direct solvers (`cholmod`, `accelerate`, `pardiso`) factorize each connected component once and then solve `cholmod_batch_size` right-hand sides at a time, so with many focal pairs on a small or medium grid they are usually faster. Their memory is dominated by fill-in in the factor, which grows faster than the grid. The right-hand-side, solution and residual buffers are `n × batch` dense matrices allocated once per component and reused across batches. Multi-column triangular solves are memory-bandwidth bound and saturate at a few dozen columns, so the default batch of 32 costs nothing in throughput over a larger one while keeping those buffers small.

Single precision

`precision = single` halves the memory of every vector and matrix. The cost is accuracy: see the note under *Convergence Checks* above for what to expect and how to check it.

7.5 Default Solvers: CG+AMG and CHOLMOD

Circuitscape ships with two solvers that work out of the box with no additional packages:

- **CG+AMG** (`solver = cg+amg`, the default) — an iterative solver using conjugate gradient with an algebraic multigrid preconditioner. This is the best choice for large grids because memory usage scales well with problem size. It also parallelizes individual pair solves across threads.
- **CHOLMOD** (`solver = cholmod`) — a direct solver using Cholesky factorization from SuiteSparse. It can be significantly faster than CG+AMG for small to medium problems, but memory usage grows quickly due to fill-in, making it impractical for very large grids. In pairwise mode it performs batched solves controlled by the `cholmod_batch_size` parameter.

7.6 GeoTIFF Support

Circuitscape reads and writes ESRI ASCII grids (`.asc`, optionally gzipped) natively and needs no extra packages for them. GeoTIFF input, and GeoTIFF output via `write_as_tif = True`, use GDAL through the optional ArchGDAL.jl package extension. Install it once and load it before running:

```
using Pkg
Pkg.add("ArchGDAL")
```

```
using ArchGDAL
using Circuitscape
Circuitscape.compute("config.ini")   # may now name .tif rasters, or set write_as_tif = True
```

Keeping GDAL optional cuts the default install by a large set of binary dependencies (GDAL, PROJ, GEOS, HDF5, NetCDF, ...) and their precompile time. A configuration that names a GeoTIFF without ArchGDAL loaded is refused before any data is read, with a message saying exactly this.

7.7 Pardiso Solver

Circuitscape supports the Pardiso direct solver as a package extension. Pardiso uses Intel MKL's sparse direct solver and can offer excellent performance on Intel hardware. To use it, first install Pardiso.jl:

```
using Pkg
Pkg.add("Pardiso")
```

Then load it before (or alongside) Circuitscape:

```
using Pardiso
using Circuitscape
Circuitscape.compute("config.ini")  # with solver = pardiso in the INI file
```

Pardiso requires double precision and will automatically switch if single precision is requested. Like CHOLMOD, it is a direct solver best suited for small to medium problem sizes, and uses the same batched solve strategy in pairwise mode.

7.8 Apple Accelerate Solver

On macOS (13.4 or later), Circuitscape can use Apple's Accelerate framework for sparse Cholesky factorization. This is a direct solver that can provide high performance on Apple Silicon hardware. To use it:

```
using AppleAccelerate
using Circuitscape
Circuitscape.compute("config.ini")  # with solver = accelerate in the INI file
```

Or install `AppleAccelerate` first:

```
using Pkg
Pkg.add("AppleAccelerate")
```

The Accelerate solver supports both single and double precision, and uses the same batched solve strategy as the CHOLMOD and Pardiso solvers.

Note

Circuitscape sets BLAS to single-threaded at startup. Its workload is predominantly sparse matrix operations which do not benefit from multi-threaded BLAS.

Part V

Calling Circuitscape from other Programs

Chapter 8

Calling Circuitscape from Other Programs

Circuitscape is a Julia package with one entry point, `Circuitscape.compute`, which takes either the path of an INI file or a dictionary of INI keys and string values. Nothing is exported, so every call is written fully qualified (`Circuitscape.compute`, `Circuitscape.init_config`); `using Circuitscape: compute` brings the bare name in if you prefer. Anything that can call Julia can therefore run Circuitscape. Output files are written next to `output_file`; `compute` also returns the result matrix (pairwise resistances with the focal node IDs in the first row and column).

8.1 From Julia

```
using Circuitscape
result = Circuitscape.compute("job.ini")
```

Or build the configuration in code, starting from the defaults:

```
using Circuitscape
cfg = Circuitscape.init_config()
cfg["habitat_file"] = "resistance_map.asc"
cfg["point_file"] = "focal_nodes.asc"
cfg["scenario"] = "pairwise"
cfg["output_file"] = "output/results.out"
cfg["write_cur_maps"] = "True"
result = Circuitscape.compute(cfg)
```

Values are the same strings you would write in an INI file (`"True"`, `"cholmod"`, `"single"`, ...); unrecognised values are errors, see *Configuration Validation* in Inputs, Outputs and Options.

To run in parallel, start Julia with threads and turn parallelism on in the configuration:

```
JULIA_NUM_THREADS=8 julia myscript.jl      # or: julia -t 8 myscript.jl
```

```
cfg["parallelize"] = "True"
```

GeoTIFF input or output (`write_as_tif = True`) needs the optional ArchGDAL package, loaded before the run:

```
using Pkg; Pkg.add("ArchGDAL")   # once
using ArchGDAL, Circuitscape
Circuitscape.compute("job_with_tifs.ini")
```

ESRI ASCII grids (`.asc`) need no extra package.

8.2 From R

Use the JuliaCall package. Set `JULIA_NUM_THREADS` *before* `julia_setup()` if you want threads; the Julia process is started once per R session.

```
install.packages("JuliaCall")          # once
library(JuliaCall)

Sys.setenv(JULIA_NUM_THREADS = "8")    # optional, before julia_setup()
julia_setup()                          # julia_setup(installJulia = TRUE) installs Julia if needed

julia_install_package_if_needed("Circuitscape")   # once
julia_library("Circuitscape")

result <- julia_call("Circuitscape.compute", "job.ini")   # returns the resistance matrix
```

`julia_call` and `julia_eval` resolve names in Julia's `Main` module, and Circuitscape exports nothing, so the function is always written `Circuitscape.compute` (alternatively run `julia_command("using Circuitscape: compute")` once and then call `julia_call("compute", ...)`).

A configuration can be built in R as a named list of strings and passed as a Julia `Dict`:

```
cfg <- julia_call("Circuitscape.init_config")
julia_assign("cfg", cfg)
julia_command('cfg["habitat_file"] = "resistance_map.asc"')
julia_command('cfg["point_file"] = "focal_nodes.asc"')
julia_command('cfg["scenario"] = "pairwise"')
julia_command('cfg["output_file"] = "output/results.out"')
julia_command('cfg["parallelize"] = "True"')
result <- julia_eval("Circuitscape.compute(cfg)")
```

For GeoTIFF rasters add `julia_install_package_if_needed("ArchGDAL")` and `julia_library("ArchGDAL")` before `julia_library("Circuitscape")`.

8.3 From Python

Use the juliacall package (`pip install juliacall`). It installs Julia on first use if none is found. Threads are set with the `PYTHON_JULIACALL_THREADS` environment variable before `juliacall` is imported.

```
import os
os.environ["PYTHON_JULIACALL_THREADS"] = "8"   # optional, before the import

from juliacall import Main as jl
```

```
jl.seval('import Pkg; Pkg.add("Circuitscape")')  # once
jl.seval("using Circuitscape")

result = jl.Circuitscape.compute("job.ini")      # Julia Matrix; np.asarray(result) converts
```

As in R, nothing is exported: functions are reached through the module object, `jl.Circuitscape.compute`, `jl.Circuitscape.init_config`.

Building the configuration in Python:

```
cfg = jl.Circuitscape.init_config()
cfg["habitat_file"] = "resistance_map.asc"
cfg["point_file"] = "focal_nodes.asc"
cfg["scenario"] = "pairwise"
cfg["output_file"] = "output/results.out"
cfg["parallelize"] = "True"
result = jl.Circuitscape.compute(cfg)
```

For GeoTIFF rasters run `jl.seval('import Pkg; Pkg.add("ArchGDAL")')` once and `jl.seval("using ArchGDAL")` before `using Circuitscape`.

8.4 Downstream packages

Omniscape.jl is the main package built on Circuitscape. It runs omnidirectional connectivity analyses by calling `Circuitscape.compute_omniscape_current` on in-memory arrays for every moving window, and is the reference for embedding Circuitscape in another Julia package. See the API Reference.

Part VI

Logging Options

Chapter 9

Logging Options

Circuitscape uses a custom `CSLogger` built on Julia's `AbstractLogger` interface. The logger is installed as the global logger when you call `Circuitscape.compute()`, based on the `log_level`, `log_file` and `suppress_messages` settings in your INI file. Every message is prefixed with a timestamp.

9.1 Log Level

`log_level` accepts `DEBUG`, `INFO` (the default), `WARNING`, `ERROR` and `CRITICAL`, in any case. `WARN` is also accepted, and `CRITICAL` is treated as `ERROR`, as in Circuitscape 4. Any other value is an error.

```
log_level = DEBUG
```

At `DEBUG`, every pair solve is logged (`Solving pair k of n`) and, when the run finishes, a timing table produced with TimerOutputs is printed. It breaks the run down into loading the raster, constructing the graph and the preconditioner or factorization, `solve linear system` and `postprocess` per task, and writing cumulative maps. This is the place to look when deciding between solvers or checking that threads are being used.

9.2 Suppressing Messages

Set `suppress_messages = True` in your INI file to suppress informational messages on the console. Warnings and errors are still displayed, and a log file, if set, still receives every message.

9.3 Logging to File

Set `log_file` in your INI file to write log messages to a file:

```
log_file = /path/to/logfile.log
```

When a log file is set, messages are written to both the console and the file. The file is created (or truncated) at the start of each run.

9.4 Disabling Log Output

To disable all informational log messages from Julia's side, use the built-in logging system:

```
using Logging
Logging.disable_logging(Logging.Info)
```

To re-enable:

```
Logging.disable_logging(Logging.Debug)
```

Part VII

Upgrading to 6.0

Chapter 10

Upgrading to 6.0

Circuitscape.jl 6.0 is the first release since 5.17.1 and collects everything merged since then. Most INI files run unchanged. This page lists the changes that can require action when upgrading.

10.1 Breaking changes

- **`compute` is no longer exported.** Nothing is: call `Circuitscape.compute(...)`, or add `using Circuitscape: compute` to keep the bare name.
- **`start` is no longer exported.** Call the interactive INI builder as `Circuitscape.start()`; a bare `start()` after `using Circuitscape` is an `UndefVarError`.
- **Configuration values are validated.** Unrecognised strings for `solver`, `scenario`, `data_type`, `precision`, `log_level` and `remove_src_or_gnd`, non-boolean values for boolean options, a missing `scenario`, input files that do not exist and a missing output directory are now errors, reported together before any data is read, instead of silently falling back to a default. Run each INI file once and fix what the message names; see *Configuration Validation* in Inputs, Outputs and Options.
- **GeoTIFF read and write require the optional ArchGDAL package.** Run `Pkg.add("ArchGDAL")` once, then `using ArchGDAL` before `using Circuitscape` whenever a job names a `.tif` or sets `write_as_tif = True`. ESRI ASCII grids need nothing extra.
- **`low_memory_mode` and `preemptive_memory_release` set to `True` are errors.** These Circuitscape 4 options were never implemented here; remove them or set them to `False`.

Everything else in 6.0 is additive or internal; see the release notes for the full list, and *What Makes a Run Fast* in On Solvers and Computation Time for the performance changes.

Part VIII

API Reference

Chapter 11

API Reference

Circuitscape exports nothing: every function, `compute` included, is reached as `Circuitscape.name` (or brought in explicitly with `using Circuitscape: compute`). The names in the **Public API** section are covered by semantic versioning: they keep working, with the documented meaning, within a major version. Anything not listed there is internal and may change in any release. Omniscape.jl depends on exactly three of these, `compute_omniscape_current`, `init_config` and `update!`, and they are stable.

11.1 Public API

`Circuitscape.compute` – Function.

```
Circuitscape.compute(path::String) -> Matrix
Circuitscape.compute(cfg::Dict{String,String}) -> Matrix
```

Run a Circuitscape job. `path` names an INI file; alternatively pass a dictionary of INI keys and string values, as returned by `Circuitscape.init_config`, with the entries you need changed. Keys that are not given take their default value.

The configuration is validated before any data is read (see `Circuitscape.validate`), written in INI form to `output_file`, and run. Output files are written next to `output_file`; the return value is the result matrix: pairwise resistances (first row and column hold the focal node IDs), one-to-all resistances per focal node, node voltages in advanced mode. Set `parallelize = True` and start Julia with several threads to run pair solves in parallel.

source

`Circuitscape.init_config` – Function.

```
init_config() -> Dict{String,String}
```

The default configuration as INI strings, one entry per key `compute` accepts. Modify the entries you need and pass the dictionary to `compute`, or convert it with `CSConfig(dict)`. File paths default to INI Builder placeholders such as `"(Browse for a resistance file)"`, which `validate` treats as unset. A few Circuitscape 4 keys (`print_timings`, `print_rusages`, `screenprint_log`, `profiler_log_file`) are present so that old INI files load without warnings; they have no effect.

source

`Circuitscape.update!` – Function.

```
update!(cfg::Dict{String,String}, new) -> cfg
```

Merge the INI keys and values in `new` (any iterable of `key => value` pairs, typically a `Dict{String,String}`) into the configuration dictionary `cfg`, overwriting existing entries. Mutates and returns `cfg`. This is how a caller layers its own settings on top of `init_config` before passing the result to `compute` or `compute_omniscape_current`; no values are parsed or checked here (that happens in `CSConfig(cfg)`).

source

`Circuitscape.compute_omniscape_current` – Function.

```
compute_omniscape_current(conductance, source, ground, cs_cfg) -> Matrix{Float64}
```

Advanced-mode solve on in-memory arrays; Omniscape's entry point for every moving-window solve. No files are read or written.

Arguments:

- `conductance::Matrix{T}`, `T <: Union{Float32, Float64}`: per-cell conductance (not resistance). Cells with a value of `0` are dropped from the graph; every other cell becomes a node connected to its eight neighbours (four if `connect_four_neighbors_only = True` in `cs_cfg`) with the average conductance of the two cells as edge weight.
- `source::Matrix{T}`: current injected at each cell, in amps; `0` where there is no source.
- `ground::Matrix{T}`: conductance to ground at each cell (`Inf` for a direct ground, `0` where there is no ground). Where a cell is both a source and a ground the source is removed (`remove_src_or_gnd = rmvsrc`).
- `cs_cfg::Dict{String,String}`: INI keys and values as from `init_config` merged with `update!`. It is parsed with the same strict rules as an INI file, so `scenario` must be set and values must be valid. Of it, only `solver`, `cholmod_batch_size` and `residual_tolerance` (solver settings) and `connect_four_neighbors_only` affect the result; `data_type`, `scenario`, `ground_file_is_resistances`, `remove_src_or_gnd`, `connect_using_avg_resistances` and every map-writing option are overridden. The arithmetic precision is `T`, the element type of `conductance`, not `cs_cfg["precision"]`; node indices are always `Int64`.

Returns a `Matrix{Float64}` of `size(conductance)`: the current through each cell (`max(inflow, outflow)` at its node), summed over every connected component that contains both a source and a ground. Cells with no node, or in components without a source or a ground, are `0`. The solve of each component is accepted only if its residual passes the gate described at `residual_tolerance`.

source

`Circuitscape.start` – Function.

```
Circuitscape.start()
```

Build an INI file interactively, step by step, and optionally run it. The builder lives in an extension on the REPL stdlib; calling `start()` loads REPL on demand, so it works from a script as well as from the prompt.

source

Configuration

A configuration is a `Dict{String,String}` of INI keys and values while it is being built (`init_config`, `update!`), and a `CSConfig` once parsed. `compute` accepts the dictionary form and does the parsing and validation itself; these functions expose the same steps for scripts that want to read, check or write INI files without running them.

`Circuitscape.parse_config` – Function.

```
parse_config(path::String) -> CSConfig
```

Read a Circuitscape INI file. Section headers are ignored; every `key = value` line is parsed on top of the defaults from `init_config`, so keys that are absent take their default value. Unrecognised values for `solver`, `scenario`, `data_type`, `precision`, `log_level`, `remove_src_or_gnd` and any boolean raise an `ArgumentError` naming the key; unknown keys produce a warning and are ignored. File existence is not checked here but in `validate`.

source

`Circuitscape.CSConfig` – Type.

```
CSConfig
```

Typed, immutable configuration for one Circuitscape run: one field per INI key, with the INI's string values parsed into enums, booleans and numbers. Built by `parse_config(path)` or `CSConfig(dict::Dict{String,String})`; every parser rejects values it does not recognise rather than falling back to a default. `Dict{String,String}(cfg)` converts back to INI strings, and `CSConfig(cfg; field = value, ...)` returns a copy with fields replaced.

Field names match the INI keys documented in the options reference. Notable defaults: `data_type = dt_raster`, `solver = st_cg_amg`, `precision = pr_double`, `cholmod_batch_size = 32`, `residual_tolerance = 0.0` (meaning automatic: `1e-4` in double, `1e-3` in single), `log_level = Logging.Info`. `scenario` has no usable default and must be set.

source

`Circuitscape.validate` – Function.

```
validate(cfg::CSConfig)
```

Fail fast, before any data is read, on configurations that cannot run: input files that do not exist, an output directory that does not exist, and options that Circuitscape.jl parses for compatibility with the Python version but has never implemented. Each message names the INI key so the user knows what to change.

source

`Circuitscape.write_config` – Function.

```
write_config(cfg::CSConfig)
write_config(cfg::Dict{String,String})
```

Write the configuration, in Circuitscape INI form, to the file named by `cfg.output_file`. `compute` calls this at the start of every run so that the options used are recorded next to the results.

source

11.2 Internals

Internal

Nothing below is part of the public API. These functions and types carry no semantic-versioning guarantee and may be renamed, changed or removed in any release. They are documented for contributors and for readers who want to follow a run through the code.

Problem geometry

After loading, a raster landscape and a network are the same problem: a symmetric Laplacian over graph nodes, its connected components, and focal nodes. The one thing that differs downstream is where a node's voltage or current goes in the output, and that is what a `Geometry` knows.

`Circuitscape.Geometry` – Type.

```
Geometry
```

How graph nodes map back onto the user's coordinates when maps are written. Once loaded, a raster landscape and a network are the same problem: a symmetric Laplacian over graph nodes, its connected components and focal nodes. The only thing that differs downstream of loading is where a node's voltage or current goes in the output, and that is what a `Geometry` knows. Output code dispatches on it rather than on the data type in the config.

source

`Circuitscape.RasterGeometry` – Type.

```
RasterGeometry(nodemap, polymap, hbmeta, cellmap)
```

Nodes are cells of a grid: `nodemap[i,j]` is the node of cell `(i,j)`, zero for cells outside the habitat, and cells of one short-circuit polygon share a node. `hbmeta` gives the grid's extent and georeferencing, `cellmap` the conductance per cell (needed to mark null cells as NODATA).

source

`Circuitscape.NetworkGeometry` – Type.

```
NetworkGeometry(nodes)
```

Nodes are the user's own node ids: row k of the matrix being solved is node `nodes[k]`. For the whole graph this is the identity; restricted to a connected component it is the component's node list. Branch coordinates live on `Cumulative`.

source

`Circuitscape.restrict` – Function.

```
restrict(geometry, component)
```

The geometry of one connected component, whose matrix rows are numbered `1:length(component)`: a raster nodemap relabelled to those local indices, or the component's node ids.

source

Problem construction and mode drivers

`load_data` reads a `RasterData` or `NetworkData`; `build_problem` turns it into a `GraphProblem` (pairwise) or an `AdvancedProblem` (advanced) with the matching geometry; `run_pairwise` and `run_advanced` are the mode drivers, and one-to-all/all-to-one build an `AdvancedProblem` per focal node and call `advanced_kernel` on it.

`Circuitscape.build_problem` – Function.

```
build_problem(data, cfg)
build_problem(T, cfg)
```

The problem `cfg` describes over the loaded (or, given `T`, freshly read) data: a `GraphProblem` in pairwise mode, an `AdvancedProblem` in advanced mode. Both are built the same way for rasters and networks; `build_graph`, `focal_nodes` and `initialize_cum` dispatch on the data type to construct the graph and its output geometry.

source

`Circuitscape.run_pairwise` – Function.

```
run_pairwise(data, cfg)
run_pairwise(problem::GraphProblem, cfg)
```

Pairwise mode: solve every pair of focal nodes and write the cumulative maps. A raster whose focal nodes are regions rebuilds the graph per pair (`pairwise_regions`); everything else solves one shared problem.

source

`Circuitscape.run_advanced` – Function.

```
run_advanced(problem::AdvancedProblem, cfg)
```

Advanced mode: one solve with the user's sources and grounds.

source

`Circuitscape.AdvancedProblem` – Type.

```
AdvancedProblem
```

An advanced-mode problem: the graph Laplacian `G`, its connected components `cc`, the `Geometry` mapping nodes to output coordinates, the source and ground vectors over the nodes (`sources`, `grounds`, with the finite grounds separated out in `finitegrounds` for the solver) and the solver. Built by `build_problem` in advanced mode and, one focal node at a time, by the one-to-all / all-to-one driver; solved by `advanced_kernel`.

source

`Circuitscape.advanced_kernel` – Function.

```
advanced_kernel(prob, cfg; check_node, name, accumulate_currents)
    -> (voltages, current_map, solver_called)
```

Solve every connected component that has both a source and a ground, and write the voltage and current maps the options ask for, suffixed `name`. `voltages` is over all graph nodes (zero in components not solved); the current map is the per-cell sum over components for a raster (empty for a network), filled when `accumulate_currents`. One-to-all / all-to-one solve one focal node at a time and pass it as `check_node` to skip every other component.

source

Pairwise driver

The pairwise driver is shared by every solver. `solve` iterates the connected components of a `GraphProblem`, calling `prepare!` once per component (an AMG preconditioner or a factorization), `pair_jobs` to enumerate the `PairJobs` of that component, and `solve_pairs!` to run them, which hands every solved pair to `handle_pair` to store the resistance and to `postprocess` to accumulate and write maps through the `Cumulative` accumulators. Only `prepare!` and `solve_pairs!` differ between the iterative and direct paths.

`Circuitscape.GraphProblem` – Type.

```
GraphProblem
```

A pairwise problem: the graph Laplacian `G`, its connected components `cc`, the focal nodes as graph indices (`points`) with the user's ids alongside (`user_points`), the pairs of user ids not to solve (`exclude_pairs`), the `Geometry` that maps nodes back to output coordinates, the `Cumulative` accumulators and the solver. Raster and network runs differ only in `geometry`. Built by `build_problem` and consumed by `solve`, which may regularize `G` in place (see `component_matrix`); a problem is solved once.

source

`Circuitscape.Cumulative` – Type.

```
Cumulative
```

Accumulators shared by every pair solve of a pairwise run, guarded by `lock`: the cumulative and maximum current grids of a raster (`cum_curr`, `max_curr`; `max_curr` is empty unless `write_max_cur_maps`), or the cumulative node and branch currents of a network (`cum_node_curr`, `cum_branch_curr`, with `coords` naming each branch and `branch_index` mapping an edge in either orientation to its slot).

source

`Circuitscape.PairJob` – Type.

```
PairJob
```

One pair of focal nodes to solve within a connected component: the two graph nodes (`src_node`, `dst_node`), their local indices in the component (`comp_i`, `comp_j`), and `src_indices` / `dst_indices`, the positions in `points` that map to each node; a focal region contributes several positions, all sharing one solve. Produced by `pair_jobs`.

source

`Circuitscape.pair_jobs` – Function.

```
pair_jobs(csub, comp, points, orig_pts, exclude, first_source_only;
          positions = point_positions(points))
```

The node pairs to solve in one component, grouped by source node, in the order the log numbers them. A pair is skipped when every combination of its focal indices is excluded. With `first_source_only` (the resistance shortcut) only pairs anchored at `csub[1]` are produced. A group is returned for every source considered, even when it has no pairs, so callers can iterate sources. `comp` must be sorted ascending, as `connected_components` returns it.

source

`Circuitscape.get_num_pairs` – Function.

```
get_num_pairs(ccs, fp, exclude_pairs, user_points = fp) -> (n, numbering)
```

Number of pairs the pairwise driver will solve without the resistance shortcut: for every connected component, the pairs of distinct focal nodes in it whose user ids are not in `exclude_pairs`. `numbering` maps each `(node, node)` pair to its position in that count, which is how the log labels `Solving pair k of n`.

source

`Circuitscape.solve` – Function.

```
solve(prob, solver, cfg, log)
```

Pairwise driver shared by every solver. Enumerates the pairs per connected component, hands them to `solve_pairs!` for the solver-specific linear algebra, and stores resistances / postprocesses maps in `handle_pair`. Only `prepare!` and `solve_pairs!` differ between the iterative and direct paths.

source

`Circuitscape.component_matrix` – Function.

```
component_matrix(G, comp)
```

The Laplacian restricted to the nodes of `comp` (sorted ascending, as `connected_components` returns them). When the component is the whole graph `G` itself is returned rather than a copy: `prepare!` then regularizes `prob.G` in place for the AMG solver, which is safe because nothing reads `prob.G` after `solve` and every `GraphProblem` is solved once.

source

`Circuitscape.prepare!` – Function.

```
prepare!(solver, matrix)
```

Per-component setup, run once before the pairs of a component are solved. Returns whatever `solve_pairs!` needs for that solver: the AMG preconditioner for `AMGSolver` (after regularizing `matrix` in place), or the factorization for a direct solver.

source

`Circuitscape.solve_pairs!` – Function.

```
solve_pairs!(handle_pair, handle, solver, matrix, groups, cfg, log, pair_label)
```

Solve every job in `groups` and call `handle_pair(job, voltages)` with the voltages referenced to the source node. The iterative path solves one pair at a time in equal-sized chunks of pairs, each task with its own preconditioner workspace; the direct path factorizes once and solves batches of right-hand sides, parallelizing the postprocessing of each batch.

source

Solvers and convergence

`Circuitscape.Solver` – Type.

```
Solver
```

Abstract supertype of the solver backends. `prepare!` and `solve_pairs!` dispatch on it; everything else in the pairwise driver is shared.

source

`Circuitscape.AMGSolver` – Type.

```
AMGSolver()
```

Iterative solver: conjugate gradient preconditioned by smoothed-aggregation algebraic multigrid (`solver = cg+amg`, the default). One preconditioner is built per connected component and shared, with a private workspace per task, by the parallel pair solves.

source

`Circuitscape.CholmodSolver` – Type.

```
CholmodSolver(bs)
```

Direct solver: sparse Cholesky factorization through SuiteSparse CHOLMOD (`solver = cholmod`), solving `bs` right-hand sides per batch. Available without extra packages, in double and single precision.

source

`Circuitscape.PardisoSolver` – Type.

```
PardisoSolver(bs)
```

Direct solver through Intel MKL Pardiso (`solver = pardiso`), `bs` right-hand sides per batch. Requires the Pardiso.jl package extension (`using Pardiso`) and double precision.

source

`Circuitscape.AccelerateSolver` – Type.

```
AccelerateSolver(bs)
```

Direct solver through Apple's Accelerate sparse Cholesky (`solver = accelerate`), `bs` right-hand sides per batch. Requires the AppleAccelerate.jl package extension (`using AppleAccelerate`) on macOS; double and single precision.

source

`Circuitscape.get_solver` – Function.

```
get_solver(cfg) -> Solver
```

The solver backend selected by `cfg.solver`, with `cfg.cholmod_batch_size` as the batch size of the direct solvers. Logs which solver is used.

source

`Circuitscape.residual_tolerance` – Function.

```
residual_tolerance(cfg, T)
```

Largest acceptable true relative residual ||Gv − b|| / ||b|| for a solve in precision T. `cfg.residual_tolerance` when the user set one; otherwise `1e-4` in double and `1e-3` in single, since Float32 round-off puts `1e-4` at the edge of what CG can reach (the fixed `1e-4` gate broke every single precision run).

source

`Circuitscape.solve_linear_system` – Function.

```
solve_linear_system(G, curr, M; tol)
```

Preconditioned CG. Krylov's stopping test is on the *preconditioned* residual `sqrt(r' M⁻¹ r)`, while the result is accepted on the true 2-norm relative residual. When the AMG preconditioner is a poor fit for a component the two diverge: CG reports success in its own norm while the true residual is still above `tol` (issue #470). Rather than abort a multi-hour job on one such solve, tighten the stopping tolerances and retry; solves that pass first time are unaffected.

source

`Circuitscape.refine_columns!` – Function.

```
refine_columns!(lhs, factor, matrix, rhs, tol, name; resid = similar(lhs))
```

Check every column of a direct solve against `tol`, applying iterative refinement with the existing factor where needed. In single precision a Cholesky solve of an ill-conditioned Laplacian lands at 1e-3..1e-2, well short of what CG reaches with its 1e-5 rtol; each step costs one triangular solve and recovers several digits. Double precision residuals are far below the target and take no steps. Shared by the CHOLMOD, Pardiso and Accelerate backends.

The residuals of the whole batch come from one sparse-times-dense product `matrix * lhs` written into `resid`, which callers with a preallocated workspace pass in; only columns that fail the check enter the per-column refinement loop.

source

Input and output

`Circuitscape.read_raster` – Function.

```
read_raster(path, T) -> (array, wkt, transform)
```

Read a single-band raster as a `nrows × ncols` matrix of T with NODATA and NaN mapped to -9999. ESRI ASCII grids (`.asc`, optionally gzipped) are parsed natively; every other format goes through GDAL.

source

`Circuitscape.write_raster` – Function.

```
write_raster(fn_prefix, array, wkt, transform, file_format)
```

Write a single-band raster. `.asc` is written natively with `writedlm` — several times faster than the GDAL AAIGrid driver, and thread-safe, so it takes no lock; `.tif` goes through GDAL under `IO_LOCK`.

source

`Circuitscape.connected_components` – Function.

```
connected_components(A::SparseMatrixCSC) -> Vector{Vector{Ti}}
```

Connected components of the graph whose edges are the nonzero off-diagonal entries of the symmetric matrix `A`, computed directly on the CSC structure. Components are ordered by their smallest vertex and vertices ascend within each — the same contract as `Graphs.connected_components(SimpleGraph(A))`, which this replaces. That path built an adjacency-list copy of `A` first: at 4M cells, 1.4 GB allocated and 458 MB retained to produce 31 MB of labels.

source